

5. JavaSpaces services ensure that operations execute atomically.
6. Objects in a JavaSpaces service are shared. Both service providers and clients can read and take objects from the JavaSpaces service.

Now we demonstrate how to build a distributed image-processing system with the JavaSpaces service (Fig. 18.16). Image processing can be a time-consuming task, especially for large images. However, we can improve performance by using JavaSpaces services to build a distributed image-processing system for applying filters to images (e.g., blur, sharpen, etc.). The client first partitions a large image into smaller pieces and writes them into a JavaSpaces service. Multiple image processors (distributed among the system) run in parallel to take the smaller images from the storage and process them by applying appropriate filters, then write the processed images back into the JavaSpaces service. The client takes the processed subimages from the JavaSpaces service and builds the complete, processed image.

SelfReview

1. (T/F) After a service provider adds an object to a JavaSpaces service, the object will always be there if no one removes it.
2. How might JavaSpaces be used to implement a chat system?

*AI*9--* 1) False. Even though the client does not execute a take operation, the object will not remain in the JavaSpaces service permanently. Each object stored in a JavaSpaces service has

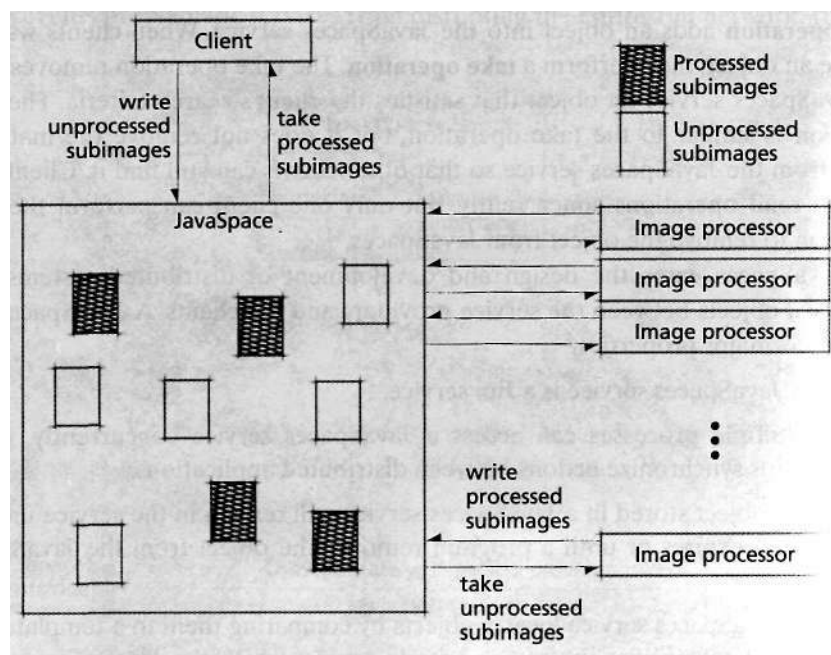


Figure 18.16 | Distributed image-processing application using JavaSpaces.

a lease; when its lease expires, the object is removed. **2)** JavaSpaces can be used to build a chat system in which the message sender writes messages to JavaSpaces and the message receiver reads or takes the message from JavaSpaces. The sender can send messages to a particular recipient or to a group of recipients.

18.7.4 Java Management Extensions (JMX)

A crucial function in any distributed system is network management. Businesses increasingly need networks that provide services which are customized to customer demands, consistently available and easily updated. As more organizations add networks and expand existing networks to increase productivity, network management becomes more difficult. The proper functionality of printers, network routers and other devices—many of which play an integral role in the productivity of a firm—is important. As more devices are networked and as networks grow larger, more problems can occur. The demand for managing distributed applications increases when business applications move into distributed environments.⁸⁶

Existing tools for managing networked devices often use both standard and proprietary protocols and management tools. The diversity of such proprietary protocols and tools makes managing a diverse network difficult. Many network management schemes are inflexible and nonautomated. This usually means that significant amounts of time and resources are spent trying to fix and operate large networks. New technologies are required that will help shift the burden of routine tasks to the network management software itself and leave special issues to the network manager.

Recent technological advances have provided network management developers with the tools necessary to develop smart agents that can assume various responsibilities. These tools allow agents to incorporate themselves into frameworks in which numerous agents interact with each other and provide a dynamic and extensible network management solution.

Java Management Extensions (JMX), developed by Sun and other companies in the network management industry, defines a component framework that enables developers to build automated, intelligent and dynamic network management solutions. JMX defines a three-level management architecture—the instrumentation level, the agent level and the manager level (Fig. 18.17). The **instrumentation level** makes any Java-based object manageable so that the management application can access and operate these objects. The **agent level** provides services for exposing the managed resources. The **manager level** gives management applications access to resources created in the instrumentation level and operates these resources via the JMX agents.⁸⁷

Self Review

1. What are the problems with many existing network-management schemes?
2. What are the three levels in JMX architecture? Briefly describe the function of each.

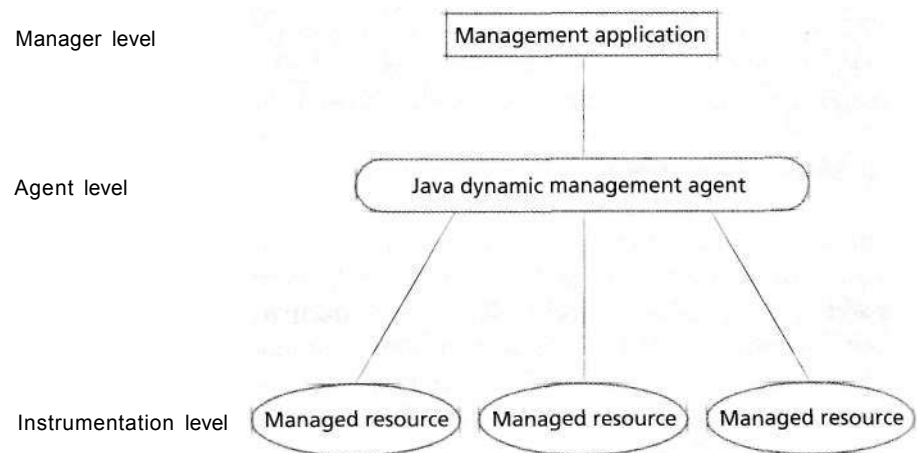


Figure 18.17 | *JMX's three-level management architecture.*

Ans: 1) Many existing network-management schemes are inflexible and nonautomated, because various networked devices may be managed using diverse protocols and tools and require large amounts of time to fix and operate in a large network. 2) The three levels are: the instrumentation level, the agent level and the manager level. The instrumentation level makes any Java-based object manageable. The agent level provides services for exposing the managed resources. The manager level gives management applications access to resources and operates these resources via the JMX agents.

18.8 Web Services

Over the past several decades, computing has evolved at an unprecedented pace. This progress impacts organizations in significant ways, forcing information-technology (IT) managers and developers to accept new computing paradigms. Innovations in programming and hardware have led to more powerful and useful technologies, including object-oriented programming, distributed computing, Internet protocols and XML (Extensible Markup Language). At the same time, organizations have learned to leverage the power of their networks and the Internet to gain competitive advantages.

Web services technology, which represents the next stage in distributed computing, will profoundly affect organizations in the future. Web services encompass a set of related standards that can enable any two computer applications to communicate and exchange data via the Internet. Many factors—including software vendors' widespread support for underlying standards—indicate that Web services will radically change IT architectures and partner relationships. Companies are already implementing Web services to facilitate a wide variety of business processes, such as application integration and business-to-business transactions.⁸⁸

For a distributed system to function correctly, application components executing on different computers throughout a network must be able to communicate. In Section 17.3, we discussed technologies such as DCOM and CORBA that enable communication among distributed components. Unfortunately, DCOM and CORBA cannot intercommunicate easily. Their components often communicate via a COM/CORBA bridge. If DCOM's and CORBA's underlying protocols change, programmers must modify the bridge to reflect the changes. Such problems have impeded distributed computing's ability to facilitate business-process integration and automation.

Web services improve distributed computing capabilities by addressing the issue of limited interoperability.⁸⁹ Web services operate using open (i.e., nonproprietary) standards, which means that Web services can, theoretically, enable any two software components to communicate—regardless of the technologies used to create the components or the platforms on which the components reside. Also, Web-services-based applications are often easier to debug, because Web services use a text-based communications protocol, rather than the (albeit faster) binary communications protocols employed by DCOM and CORBA. Organizations are implementing Web services to improve communication between DCOM and CORBA components and to create standards-based distributed computing systems. Thus, Web services will help organizations achieve the goals of distributed computing and do so more economically.⁹⁰

The industry's experience with interoperability problems led to the development of open standards for Web services technologies, in an effort to enable cross-platform communication. The primary standard used in Web services is **Extensible Markup Language (XML)**, a language for marking up data so that information can be exchanged between applications and platforms. Microsoft and DevelopMentor developed the **Simple Object Access Protocol (SOAP)** as a messaging protocol for transporting information and instructions between Web services, using XML as a foundation for the protocol. Two other Web services specifications—**Web Services Description Language (WSDL)** and **Universal Description, Discovery and Integration (UDDI)**—are also based on XML. WSDL provides a standard method of describing Web services and their specific capabilities; UDDI defines XML-based rules for building directories in which companies advertise themselves and their Web services.⁹¹

Open standards enable businesses (running different platforms) to communicate and transfer data without designing costly platform-specific software. Web services improve collaborative software development by allowing developers to create applications by combining code written in any language on any platform. Also, Web services promote modular programming. Each specific function in an application can be exposed as a separate Web service. Individuals or businesses can create their own unique applications by mixing and matching Web services that provide the functionality they need. Modularization is less error prone and promotes software reuse.

A Web service can be as trivial as multiplying two numbers together or as complex as the functions carried out by an entire customer relationship management (CRM) software system. For example, tracking the location and status of a FedEx package is a Web service (fedex.com/us/tracking).

Some e-commerce Web sites allow independent developers to harness the power of their sites' technologies by exposing certain functions as Web services. For example, online retailer Amazon.com allows developers to build online stores that search its product databases and display detailed product information via Amazon.com Web Services (www.amazon.com/gp/aws/landing.html). The Google search engine also can be integrated with other functionality through the Google Web APIs (www.google.com/apis), which connect to Google's indices of Web sites using Web services. Amazon.com and Google provide access to their sites' features through SOAP and other standard protocols in exchange for increased exposure.

SelfReview

1. How are Web services more portable than DCOM?
2. Can a client application written in Java access a Web service written in C#? Why?

Ans: 1) Web services provide interoperability, platform independence and language independence. 2) Yes, a client application written in Java can access a Web service written in C#, because Web services technologies use open standards such as XML, SOAP, WSDL and UDDI.

18.8.1 Microsoft's .NET Platform

Microsoft introduced the term Web services during the June 2000 launch of its .NET initiative. Today, Microsoft is one of the dominant companies in the Web services market. The .NET initiative includes the Visual Studio .NET integrated development environment which enables programmers to develop Web services in a variety of languages, including C++, C# and Visual Basic .NET. However, .NET technologies are available only for Windows 2000 and XP.⁹²

.NET Web services, which are central to the .NET initiative, extend the concept of software reuse to the Internet by allowing developers to reuse software components that reside on other machines or platforms. Employing Web services as reusable building blocks, programmers can concentrate on their specialties without having to implement every component of an application. For example, a company developing an e-commerce application can subscribe to Web services that process payments and authenticate users. This enables programmers to focus on other, more unique aspects of the e-commerce application.

In .NET, a Web service is an application stored on one machine that can be accessed by another machine over a network. In its simplest form, a Web service created in .NET is a class, or a logical grouping of methods and data that simplifies program organization. Methods are defined within a class to perform tasks and return information when their tasks are complete. .NET Web service classes contain

certain methods (called **Web service methods**) that are specified as part of the Web service. These methods can be invoked remotely using RPC.

Self Review

1. Can .NET technologies be used on Linux?
2. How have .NET Web services extended the concept of software reuse to the Internet?

Ans: 1) No. .NET technologies are available only for Windows 2000 and XP. 2) .NET Web services allow developers to reuse software components that reside on other machines or platforms.

18.8.2 Sun Microsystems and the Sun ONE Platform

Sun Microsystems' Web services strategy is based on the **Sun Open Net Environment (Sun ONE)**, which consists of three components—a vision, an architecture and a conceptual model for developing standards-based software.⁹³

The Sun ONE vision incorporates a model for software development, in which critical business information and applications are available at any time to any type of device, including cell phones and PDAs. Sun ONE's goal is to help developers create networks of distributed applications or Web services that are highly reliable and promote the reuse of components and services.⁹⁴

The Sun ONE architecture is designed to be scalable to ensure reliable access to services. Scalability is crucial; as new technologies and new components are added to systems, more demands are placed on system resources, potentially degrading service.⁹⁵

The Sun ONE platform is comprised of three products: the Solaris™ Operating Environment, the Infrastructure Software and the Sun ONE Studio. The Infrastructure Software includes the Sun ONE Directory Server and the Sun ONE Portal Server, which offer user authentication and personalization. Other Infrastructure Software capabilities include scheduling management, billing and communication. Sun ONE allows programmers to deploy Web services using third-party products. By integrating disparate products, programmers can develop Web services infrastructures that best suit their companies' requirements. Sun ONE incorporates support for open standards, including XML, SOAP, UDDI and WSDL, to help ensure high levels of interoperability and system integration.⁹⁶

Sun ONE promotes the notion that a company's Data, Applications, Reports and Transactions, which compose the conceptual DART model, can be published as services online.⁹⁷ Using the DART model, companies can organize business applications and processes that involve data, applications, reports and transactions, so that programmers can map business elements to corresponding services.

Self Review

1. How is Sun ONE more portable than .NET?
2. What does infrastructure software do in Sun ONE?

Ans: 1) .NET is available only on Windows, while Sun ONE is available for many platforms. **2)** Infrastructure software offers user authentication and personalization. Infrastructure software is also capable of scheduling management, billing and communication.

www.faqs.org/ftp/rfc/pdf/rfc1094.txt.pdf
PDF version of the Network File System version 2 protocol specification.

www.connectathon.org/nfsv3.pdf
PDF version of the Network File System version 3 protocol specification.

www.ietf.org/rfc/rfc2054.txt
Text version of the WebNFS client specification.

www.ietf.org/rfc/rfc3010.txt
Text version of the Network File System version 4 protocol specification.

www.ietf.org/rfc/rfc2203.txt
Text version of the RPCSEC_GSS protocol specification.

www.cs.berkeley.edu/projects/sprite/retrospective.html
Brief retrospective on the Sprite Network Operating System.

www.es.jamstec.go.jp/esc/eng/index.html
Earth simulator home page.

www.distributed.net/pressroom/presskit.php
Provides news and articles for distributed system projects.

library.thinkquest.org/C007645/english/1-parallel.htm
Compares distributed computing and parallel computing and introduces clustering.

www.linux-mag.com/2000-10/clustering_01.html
Discusses Linux clustering in detail.

www.jini.org
The Jini technology site; provides links for downloading the Jini technology implementation.

www.sun.com/software/jini/specs/js2_0.pdf
Provides the JavaSpaces service specification.

java.sun.com/products/HavaManagement/wp/
Contains Java Management Extensions (JMX) White Paper.

www.jxta.org
Official Web site for Project JXTA; contains the newest downloads of the source code and opportunities to participate in developing JXTA.

www.openp2p.com
This O'Reilly Network online resource provides articles and links related to peer-to-peer technologies.

java.sun.com/j2ee/corba/
Discusses CORBA technology and the Java platform.

java.sun.com/j2ee
Provides reference material for all Java 2 Enterprise Edition technologies, including servlets and JavaServer Pages.

jakarta.apache.org
The Apache Jakarta Project site provides resources for servlets and JavaServer Pages and the reference implementation of these technologies.

www.webservices.org
The Web Services Community Portal contains the latest news regarding Web services and Web services vendors. There is also a collection of articles and papers relating to Web services technologies.

www.microsoft.com/net
Microsoft's .NET site provides .NET resources, including product information, news, events and downloads.

www.sun.com/sunone
Sun Microsystems' Sun ONE initiative site explains the Open Net Environment and provides news about the products and its users.

www.xmethods.com
Lists free, publicly available Web services and provides links for tutorials and implementations of Web services.

Summary

Networked file systems allow clients to access files stored on remote computers. Distributed file systems, which are the focus of this chapter, are special examples of networked file systems that allow transparent access to remote files.

A distributed file server can be either stateful or stateless. In a stateful system, the server keeps state infor-

mation of the client requests so that subsequent access to the file is easier. In a stateless system, the client must specify which file to access in each request.

Distributed file systems provide the illusion of transparency. Complete file location transparency means that the user is unaware of the physical location of a file within a

distributed file system; the user just sees a global file system. Many distributed systems implement client caching to avoid the overhead of multiple RPCs. Clients keep a local copy of a file and flush it to the server from time to time. Because there are multiple copies of the same file, files can become inconsistent. Distributed file systems are designed to share information among large groups of computers. To reach this goal new computers should be able to be added to the system easily. Distributed file systems should be scalable. There are two primary security concerns in distributed file systems: ensuring secure communications and guaranteeing correct file access permissions.

NFS (Network File System) versions 2 and version 3 assume a stateless server implementation, which makes fault tolerance easier to implement than with a stateful server. With stateless servers, if the server crashes, the client can simply retry until the server responds. NFS-4 is stateful. The stateful server allows faster access to files. However, if the server crashes, all the state information of the client is lost, so the client needs to rebuild its state on the server before retrying.

Client-side caching is essential to efficient distributed file system operation. NFS-4 extends the client-caching scheme through delegation, which allows the server to temporarily transfer the control of a file to a client. When the server grants a read delegation of a particular file to the client, then no other client can write to that file. When the server grants a write delegation of a particular file to the client, then no other client can read or write to that file. If another client requests a file that has been delegated, the server will revoke the delegation and request that the original client flush the file to disk.

AFS (Andrew File System) is a global file system that appears as a branch of a traditional UNIX file system at each workstation. AFS is completely location transparent and provides a high degree of availability, which means the files are always available regardless of the user's location. AFS is based on the client-server model and relies on RPCs for communication.

The Sprite file system has the perceived form of a UNIX file system, and every client has the exact same view of the hierarchy. It goes a step further in UNIX file system emulation by allowing transparent remote access to I/O devices (which are represented as files in UNIX). In Sprite, both the client and the server keep a cache. To open a file, the client first checks its cache, then makes a request to its backing store (which is generally a server). If the server is unable to satisfy the request from its cache, it reads the data from disk. Both caches retain a copy of the data when

it is returned to the client. When a client writes a block, it writes into its cache. Sprite flushes updated pages to the server every 30 seconds.

Clustering involves interconnecting nodes (single-processor computers or multiprocessor computers) within a high-speed LAN to function as a single parallel computer. The set of nodes that forms the single parallel machine is called a cluster. Clustering enables multiple computers to work together to solve large and complex problems such as weather prediction.

There are three major types of clusters: high-performance clusters, high-availability clusters and load-balancing clusters. In a high-performance cluster, all the nodes in the cluster work to improve performance. In a high-availability cluster, only some of the nodes in the cluster are working while others act as backups. If working nodes or their components fail, the backup nodes immediately start running and take over the jobs that were being executed on the failed nodes, without interrupting service. In a load-balancing cluster, a particular node works as a load balancer to distribute the load to a set of nodes so that all hardware is efficiently utilized.

P2P (peer-to-peer) applications are different from client/server applications. Instead of segregating computers by function, all computers act as both clients and servers. Each peer has the ability to discover and communicate with other peers. Peers may share resources (such as large multimedia files) with others. P2P applications can be implemented in two forms: centralized and decentralized. A centralized P2P application uses a server that connects to each peer. Centralized P2P applications exemplify the client/server relationship. A pure P2P application, also called a decentralized P2P application, does not have a server and therefore does not suffer from the same deficiencies as applications that depend on servers.

Distributed searches make networks more robust by removing single points of failure, such as servers. Not only can peers find information in this way, but they can search for other peers via distributed searches.

Grid computing links computational resources that are distributed over the wide area network (such as computers, data storages and scientific devices) to solve complex problems. Resources in grid computing are distributed, and users can access these resources transparently without knowing where they are located. Grid computing emphasizes public collaboration—individuals and research institutes coordinate resources that are not subject to a centralized control to deliver qualities of service. Grid computing has the same advantage as clustering—high per-

formance, achieved by cost-effectively using spare computer power and collaborating resources. However, grid computing requires advanced software to manage distributed computing tasks efficiently and reliably.

A Java servlet extends the functionality of a server, most frequently a Web server. Using special syntax, JSP allows Web-page programmers to create pages that use encapsulated Java functionality and even to write scriptlets (Java code embedded in a JSP) of actual Java code directly in the page.

JSPs are generally used when most of the content sent to the client is static text and markup and only a small portion of the content is generated dynamically with Java code. Servlets are commonly used when a small portion of the content sent to the client is static text or markup. In fact, some servlets do not produce content. Rather, they perform a task on behalf of the client (such as a database query), then invoke other servlets or JSPs to provide a response.

Project JXTA strives to create a standard, low-level platform and language-independent protocol that promotes interoperability among peer-to-peer applications. JXTA provides a foundation on which developers can build any type of P2P application.

Jini—a framework for building reliable and fault-tolerant distributed systems with existing Java technologies—extends this idea of providing services beyond computer-

based networks and into home-based networks. The JavaSpaces service provides distributed, shared storage (and shared memory) for Java objects and enables Java objects to communicate, share objects and coordinate tasks using the storage. Any Java-compatible client can put shared objects into the storage.

Java Management Extensions (JMX) defines a component framework that enables developers to build automated, intelligent and dynamic network-management solutions.

Web services is an emerging area of distributed computing. Web services encompass a set of related standards that can enable any two computer applications to communicate and exchange data via the Internet. Web services can enable any two software components to communicate—regardless of the technologies used to create the components or the platforms on which the components reside.

Web services improve collaborative software development by allowing developers to create applications by combining code written in any language on any platform. Also, Web services promote modular programming. Each specific function in an application can be exposed as a separate Web service. With separate Web service components, individuals or businesses can create their own unique applications by mixing and matching Web services that provide the functionality they need. Such modularization is less error prone and promotes software reuse

Key Terms

advertisement (in JXTA)—XML document formatted according to JXTA specifications that is used by a peer to advertise itself and notify others of its existence.

agent level (in JMX)—JMX level that provides services for exposing the managed resources.

Andrew File System (AFS)—Scalable distributed file system that would grow to support a large community while being secure. AFS is a global file system that appears as a branch of a traditional UNIX file system at each workstation. AFS is completely location transparent and provides a high degree of availability.

Apache Software Foundation—Provides open-source software, such as Tomcat, the official reference implementation of JSP and servlet specifications.

application layer (in grid computing)—Contains applications that use lower-level layers to access the distributed resources.

available volume storage group (AVSG) (in Coda)—Members of the VSG with which the client can communicate.

Beowulf cluster—Linux clustering solution, which is a high-performance cluster. A Beowulf cluster may contain several nodes or several hundred. Theoretically, all nodes have Linux installed as their operating system and are interconnected with high-speed Ethernet. Usually, all the nodes in the cluster are connected within a single room to form a supercomputer.

callback (in AFS)—Sent by the server to notify the client that the cached file is modified.

cell (in AFS-3)—Unit in AFS-3, which preserves namespace continuity while allowing different systems administrators to oversee each cell.

centralized P2P application—Uses a server that connects to each peer.

client caching—Clients keep local copies of files and flush them to the server after having modified the files.

client modification log (CML) (in Coda)—Log which is updated to reflect file changes on disk.

cluster—Set of nodes that forms what appears to be a single parallel machine.

clustering—Interconnection of nodes within a high-speed LAN so that they function as a single parallel computer.

Coda Optimistic Protocol—Protocol used by Coda clients to write a copy of the file to each of the members of the AVSG, which provides a consistent view of a file within an AVSG.

collective layer (in grid computing)—Layer responsible for coordinating distributed resources, such as scheduling a task to analyze data received from a scientific device.

concurrent write-sharing—Occurs when two clients modify cached copies of the same file.

connectivity layer (in grid computing)—Layer that carries out reliable and secure transactions between distributed resources.

decentralized peer-to-peer application—Also called a pure peer-to-peer application. It does not have a server and therefore does not suffer from the same deficiencies as applications that depend on servers.

delegation (in NFS)—Allows the server to temporarily transfer the control of a file to a client. When the server grants a read delegation of a particular file, then no other clients can write to that file, but they can read it. When the server grants a write delegation of a particular file to the client, then no other clients can read or write that file.

discovery (in Jini)—Process of finding the lookup services and obtaining references to them.

distributed file system—Special examples of networked file systems that allow transparent access to remote files.

distributed search—Searching technology used in peer-to-peer applications to make networks more robust by removing single points of failure, such as servers. In a distributed search, if a peer cannot answer the client's request, the peer forwards the request to its directly connected peers and the search is distributed to the entire peer-to-peer network.

domain (in Sprite file system)—Unit that represents a portion of the global file hierarchy and is stored at one server.

emulation stage—When a Coda client becomes disconnected, it is said to enter the emulation stage. During this stage all file read requests are satisfied from cache. Write requests during the emulation stage occur in two steps. First, the file is updated on disk. Second, a log called the client modification log (CML) is updated to reflect file changes.

export local file—Performed by an NFS server to make the local directory of files available to the remote client via the mount protocol.

Extensible Markup Language (XML)—Language for marking up data so that information can be exchanged between applications and platforms.

fabric layer (in grid computing)—Layer that accesses physical resources, such as disks.

file handle—Identifies a file on the file server with file type, location and access permissions.

file identifiers (fids) (in AFS)—Entity that specifies a volume, an index within a volume and an identifier to guarantee object uniqueness within a volume.

grid computing—Links computational resources that are distributed over the wide area network (such as computers, data storages and scientific devices) to solve complex problems.

head node (in a Beowulf cluster)—Node, also called master node, that acts as a server to distribute the workload, control access to the cluster and handle the shared resources.

high-availability cluster—Cluster in which only some of the nodes are working while other nodes act as backups. The goal of a high-availability cluster is to stay up all the time.

high-performance cluster—Cluster in which all the nodes work to achieve maximum performance.

hoarding stage (in Coda)—Stage that clients enter when they are connected to Coda. In this stage, clients prepare for a possible disconnection from the system by caching any requested file.

idempotent request—A requested operation that if performed several times will return the same result, so it is acceptable to perform the same operation twice.

instrumentation level (in JMX)—Makes any Java-based object manageable so that the management application can access and operate these objects.

invalidate—To invalidate a file, the client checks the time stamp of the copy of the file on the server. If this copy has been updated since the client last copied the file, then the client must download the latest version. If the server copy has not been updated since the client copied it, then the client can work on its cached copy.

Jakarta project—Provides commercial-quality server solutions based on the Java platform that are developed in an open and cooperative fashion.

Java Community Process—Open organization that focus on developing Java technology specifications, including servlet and JavaServer Pages.

Java Management Extensions (JMX)—Developed by Sun and network management industry leaders, which defines a component framework that enables developers to build

automated, intelligent and dynamic network management solutions.

JavaServer Pages (JSP)—Allows Web-page programmers to create pages that use encapsulated Java functionality and even to write scriptlets of actual Java code directly in the page.

JavaSpaces—Jini service that implements a simple, high-level architecture for building distributed systems. The JavaSpaces service provides distributed, shared storage (and shared memory) for Java objects and enables Java objects to communicate, share objects and coordinate tasks using the storage.

Jini—Framework for building reliable and fault-tolerant distributed systems with existing Java technologies. Jini extends the idea of providing services beyond industry-based computer networks and into home-based networks.

JXTA—Project created at Sun Microsystems, Inc., which creates a standard, low-level, platform and language-independent protocol that promotes interoperability among peer-to-peer applications.

lease—Agreement between the client and server for controlling file locks.

load-balancing cluster—Cluster in which one particular node works as a load balancer to distribute the load to a set of nodes, so that all hardware is efficiently utilized.

load balancer—Node in a load-balancing cluster that distributes the workload (such as thousands of requests from the clients) to a set of nodes so that all hardware is efficiently utilized.

lookup service—Heart of the Jini architecture, which maintains information about available Jini services and enables clients to discover and use them.

manager level (in JMX)—Level that gives a management application access to managed resources (created in the instrumentation level) and operates these resources via the JMX agents.

master node (in a Beowulf cluster)—Node also known as the head node, that acts as a server to distribute the workload, control the access to the cluster and handle the shared resources.

multicomputer system—System in which processors do not share a common memory or bus. Each processor has access to its own resources. These independent processors are connected in a network to operate cooperatively to form a multicomputer system.

Network File System (NFS)—Current de facto standard for network file sharing, natively supported in most varieties of UNIX (and many other operating systems) with client

and server software available for other common platforms.

network file system—System that allows clients to access files on remote computers. Network file systems do not provide location transparency as distributed file systems do.

nonvolatile storage—The contents of nonvolatile storage are not lost when the machine loses power or is powered off.

peer—Single computer in a peer-to-peer system.

peer discovery—Finding peers in a peer-to-peer application.

peer group—Logical construct that represents a set of peers. A peer group is one of the basic types of entities in a network built with the JXTA protocols.

peer-to-peer (P2P) application—Distributes processing responsibilities and information to many computers, thus reclaiming otherwise wasted computing power and storage space and eliminating central points of failure. In a peer-to-peer system, each peer performs both client and server functions.

pipe (in JXTA)—Virtual communication channel that connects two or more peers for sending and receiving messages among peers.

preferred server (PS) (in Coda)—Member of the AVSG that provides copies of files for Venus.

prefix table (in Sprite)—Table that stores domain information to aid in file lookups. Each entry in the table represents a separate domain and consists of the absolute path to the root directory within the domain, the server which houses the domain and a token that identifies the domain.

pure peer-to-peer application—Also called a decentralized peer-to-peer application. It does not have a server and therefore does not suffer from the same deficiencies as applications that depend on servers.

read operation (in JavaSpaces)—Operation that is similar to the take operation, but does not remove the object from the JavaSpaces service so that other clients can still find it.

reintegration stage (in Coda)—Stage right after the client reconnects to the system during which Venus asynchronously updates the server using the CML.

resources layer (in grid computing)—Layer that enables applications to query and share a resource.

safe asynchronous write (in NFS-3)—Allows a server to return before a write has been completed.

scriptlet—Java code embedded in a JSP.

sequential write-sharing—Occurs when one client modifies a file, then another client tries to read or write its own cached copy of the file. Sequential write-sharing introduces cache inconsistency.

servlet—Enhances the functionality of Web servers to provide capabilities such as secure access to Web sites, interacting with databases on behalf of a client, dynamically generating custom documents to be displayed by browsers and maintaining unique session information for each client.

servlet container—Server that executes a servlet. Also known as servlet engine.

Simple Object Access Protocol (SOAP)—Messaging protocol for transporting information and instructions between Web services, using XML as a foundation for the protocol.

slave node (in a Beowulf cluster)—Beowulf cluster node that is not a head node.

Sprite—Distributed operating system whose goal is transparency and complete consistency.

stateful server—Keeps state information of the client requests—such as the file name, a pointer to the file and the current position in the file—so that the subsequent access to the file is easier and faster.

stateless server—The server does not keep state information of the client requests, so the client must specify which file to access in each request.

Sun Open Net Environment (Sun ONE)—Consists of three components—a vision, an architecture and a conceptual model for developing standards-based software.

take operation (in JavaSpaces)—Removes from the JavaSpaces service an object that matches the given criteria. Take operations, together with write and read operations, allow distributed applications to dynamically exchange objects within JavaSpaces services.

thin client—Application that requires minimal client-side support.

Tomcat—Official reference implementation of the JSP and servlet standards.

Universal Description, Discovery and Integration (UDDI)—Defines XML-based rules for building directories in

which companies advertise themselves and their Web services.

update set (in Coda)—Specifies, to each member of the AVSG, the members of the AVSG that have successfully performed the write.

Venus (in AFS)—User-level process that interacts with the Vice processes run on distributed file servers to govern distributed file access.

virtual file system (VFS)—Provides the abstraction of a common file system at each client and is responsible for all distributed file operations.

Vice (in AFS)—Entity that governs distributed file access in AFS.

volume (in AFS-2)—Introduced in AFS-2 to manage subtrees. Volumes are primarily of administrative value, allowing replication and isolation of certain subtrees, and are therefore transparent to users.

volume storage group (VSG) (in Coda)—Volumes are logical pieces of the file system and are replicated physically across multiple file servers. Servers that hold the same volume are known as a volume storage group (VSG).

WebNFS—Allows NFS clients to access WebNFS-enabled servers with a minimum of protocol overhead. Marketed as the file system for the Web, WebNFS is designed to improve NFS functionality and performance over wide area Internet and intranets.

Web service—Technology that encompasses a set of related standards that can enable any two computer applications to communicate and exchange data via the Internet.

Web Services Description Language (WSDL)—Provides a standard method of describing Web services and their specific capabilities.

Web service methods (in .NET)—Methods contained in a .NET Web service class.

write operation (in JavaSpaces)—Operation that adds an object into the JavaSpaces service.

Exercises

18.1 Explain the benefits and drawbacks of client caching in distributed file systems.

18.2 What are the primary concerns of implementing distributed file systems?

18.3 A file server can be either stateful or stateless. What are the advantages and disadvantages of each implementation?

18.4 How can distributed file systems ensure cache consistency?

18.5 Is NFS-4 more secure than NFS-2 and NFS-3? How?

18.6 In AFS-1, each time the VFS tried to open a cached file it would have Venus contact the server to make sure that the file was still valid. What is the problem with this approach? How do AFS-2 and AFS-3 solve this problem?

18.7 Explain how Coda provides fault tolerance.

18.8 Describe the three stages Coda clients go through when reading and writing to a file on the Coda server.

18.9 How does Sprite handle server crashes?

18.10 Describe the differences between high-performance clusters, high-availability clusters and load-balance clusters. Give examples of each of these cluster types in use.

18.11 List the essential elements for building a Beowulf cluster.

18.12 Compare centralized and pure peer-to-peer applications. What are the advantages and disadvantages of each approach?

18.13 Given the peer network of Fig. 18.18, apply the distributed search mechanism used in Gnutella to find the requested resource which is on P₇. Assume the search starts at P₁.

Suggested Projects

18.19 Research transparency in distributed file systems. What research is currently being done in that field?

18.20 Research client caching in distributed file systems. What research is currently being done in that field?

18.14 Given the peer network as shown in Fig. 18.18, apply the distributed search mechanism used in Freenet to find the requested resource, which is on P₇. Assume the search starts at P₁.

18.15 Describe the three basic types of entities of JXTA.

18.16 Explain why the Jini lookup service is the heart of the Jini architecture.

18.17 When a service provider registers a service with a Jini lookup service, is the registration permanent? What is the benefit of this design?

18.18 What problems do DCOM and CORBA have? How can Web services solve this problem?

18.21 Prepare a research paper for Web services. How is it used?

Distributed file systems play an important role in distributed systems.^{98, 99} Examples of distributed file systems are NFS,^{136, 101, 102, 103, 104, 105} AFS,^{106, 107} Coda¹⁰⁸ and Sprite.^{109, 110, 111, 112} Clustering enables multiple computers to work together to solve large and complex problems.^{113, 114} The most popular clustering technologies are Beowulf and Windows Server 2003 clusters.^{115, 116} The peer-to-peer distributed computing model is another architecture for building distributed systems.^{117, 118, 119} Peer-to-peer applications use a distributed search to find a peer in the network.^{120, 121} Sun provides several technologies for developing distributed computing systems with servlets and JavaServer Pages (JSP),^{122, 123, 124, 125} including Jini,^{126, 127} JavaSpaces¹²⁸ and

JMX.¹²⁹ Web services is an emerging area of distributed computing. Web services encompass a set of related standards that can enable any two computer applications to communicate and exchange data via the Internet. Web services can enable any two software components to communicate—regardless of the technologies used to create the components or the platforms on which the components reside.^{130, 131, 132, 133}

Web services is one of the most recent distributed systems architectures.^{134, 135} Distributed systems exist in many forms—distributed object-based systems, distributed file systems, distributed document-based systems and distributed coordination-based systems. Tanenbaum and Steen discuss each of these types

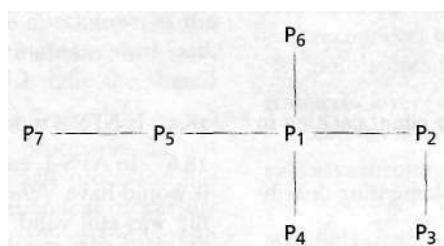


Figure 18.18 | P2P system with seven peers.

of distributed systems.¹³⁶ Both centralized and decentralized P2P applications have scalability limitations. Grid computing

enables the creation of a large-scale infrastructure used to share computing power and data resources.^{137, 138}

Works Cited

1. Tanenbaum, A., and R. Renesse, "Distributed Operating Systems," *ACM Computing Surveys*, Vol. 17, No. 4, December 1985, pp. 419-470.
2. Levy, E., and A. Silberschatz, "Distributed File Systems: Concepts and Examples," *ACM Computing Surveys*, Vol. 22, No. 4, December 1990, pp. 321-374.
3. Howard, J. H., et al, "Scale and Performance in a Distributed File System," *ACM Transactions on Computer Systems*, Vol. 6, No. 1, February 1988, pp. 51-81.
4. Andrade, H., "Distributed File Systems", <www.cs.umd.edu/~hcm/818g/>.
5. Tanenbaum, A., and R. Renesse, "Distributed Operating Systems," *ACM Computing Surveys*, Vol. 17, No. 4, December 1985, pp. 429-430.
6. "Difference Between Fault Tolerance and High-Availability," <aix.caspar.it/ibm/web/pssp-3.5/planv2/am0s0mst38.html>.
7. Froese, K. W., and R. B. Bunt, "The Effect of Client Caching on File Server Workloads," *Proceedings of the Twenty-Ninth Hawaii International Conference on System Sciences*, Wailea, HI, January 1996, pp. 15-159.
8. Tanenbaum, A., and R. Renesse, "Distributed Operating Systems," *ACM Computing Surveys*, Vol. 17, No. 4, December 1985, pp. 427-429.
9. Sandberg, R., "The Sun Network Filesystem: Design, Implementation and Experience", *Proceedings of The USENIX Conference*, 1986, pp. 300-314, <www.cs.ucf.edu/~eurip/cop6614/sandbergnfs.pdf>.
10. Nowicki, B., "NFS: Network File System Protocol Specification," March 1989, <www.ietf.org/rfc/rfc1094.txt>.
11. Callaghan, B., "NFS Version 3 Protocol Specification," <www.ietf.org/rfc/rfc1813.txt>, June 1995.
12. Network File Systems: Version 2 Protocol Specification, <docs.freebsd.org/44doc/psd/27.nfsrhc/paper.pdf>.
13. Callaghan, B., "WebNFS Client Specification," October 1996, <www.ietf.org/rfc/rfc2054.txt>.
14. Shepler, S., "RFC 2624-NFS Version 4 Design Considerations," June 1999, <www.faqs.org/rfcs/rfc2624.html>.
15. Pawlowski, B., et al., "The NFS Version 4 Protocol," <www.netapp.com/tech_library/3085.html>.
16. Shepler, S., et al., "NFS Version 4 Protocol," December, 2000, <www.ietf.org/rfc/rfc3010.txt>.
17. Eisler, M., "NFS Version 4 Security," December 1998, <www.ietf.org/proceedings/98dec/slides/nfsv4-eisier-98dec.pdf>.
18. Satyanarayanan, M., "Scalable, Secure and Highly Available Distributed File Access," *IEEE Computer*, May 1990, p. 9.
19. Morris, J. H., et al., "Andrew: A Distributed Personal Computing Environment," *Communications of the ACM*, Vol. 29, No. 3, March 1986, pp. 186-187.
20. Satyanarayanan, M., "Scalable, Secure and Highly Available Distributed File Access," *IEEE Computer*, May 1990, p. 10.
21. Satyanarayanan, M., "Scalable, Secure and Highly Available Distributed File Access," *IEEE Computer*, May 1990, pp. 11-12.
22. Morris, J. H., et al., "Andrew: A Distributed Personal Computing Environment," *Communications of the ACM*, Vol. 29, No. 3, March 1986, p. 193.
23. Satyanarayanan, M., "Scalable, Secure and Highly Available Distributed File Access," *IEEE Computer*, May 1990, p. 12.
24. Satyanarayanan, M., "Scalable, Secure and Highly Available Distributed File Access," *IEEE Computer*, May 1990, pp. 13-15.
25. Coda credits, <www.mit.edu/afs/sipb/project/coda/src/coda/CREDITS>.
26. Satyanarayanan, M., "Scalable, Secure and Highly Available Distributed File Access," *IEEE Computer*, May 1990, p. 15.
27. Satyanarayanan, M., "The Evolution of Coda," *ACM Transactions on Computer Systems*, Vol. 20, No. 2, May 2002, p. 89.
28. Satyanarayanan, M., "The Evolution of Coda," *ACM Transactions on Computer Systems*, Vol. 20, No. 2, May 2002, pp. 89-90.
29. Satyanarayanan, M., "The Evolution of Coda," *ACM Transactions on Computer Systems*, Vol. 20, No. 2, May 2002, pp. 90-91.
30. Satyanarayanan, M., "The Evolution of Coda," *ACM Transactions on Computer Systems*, Vol. 20, No. 2, May 2002, pp. 94-95.
31. Ousterhout, J., "A Brief Retrospective on the Sprite Network Operating System," <www.cs.berkeley.edu/projects/sprite/retrospective.html>.
32. Ousterhout, X, et al., "The Sprite Network Operating System," *IEEE Computer*, February 1988, p. 28.
33. Ousterhout, J., et al., "The Sprite Network Operating System," *IEEE Computer*, February 1988, pp. 28-30.
34. Ousterhout, J., et al, "The Sprite Network Operating System," *IEEE Computer*, February 1988, pp. 30-31.
35. Nelson, M., "Caching in the Sprite Network File System," *ACM Transactions on Computer Surveys*, Vol. 6, No. 1, February 1988, pp. 134-154.
36. Baker, M., and J. Ousterhout, "Availability in the Sprite Distributed File System," *Proceedings of the 4th workshop on ACM SIGOPS European workshop*, September 1990, pp. 1-4.
37. McGregor, J. D., and A. M. Riehl, "The Future of High Performance Super Computers in Science and Engineering," *Communications of the ACM*, Vol. 32, No. 9, September 1989, p. 1092.
38. Earth Simulator Performance, <www.es.jamstec.go.jp/esc/eng/ES/performance.html>.

- 39- Lyman, J., "The Supercomputing Speed Barrier," *Newsfactor Special Report*, September 13, 2002, <www.techextreme.com/perl/story/19391.html>.
40. Johnson, G., "At Los Alamos, Two Visions of Supercomputing," *New York Times*, June 25, 2002, <www.nytimes.com/2002/06/25/science/physical/25C0MP.html?pagewanted=print&position=bottom>.
41. McNett, D., "Press Information," <www.distributed.net/pressroom/presskit.html>.
42. Press Release: "Intel and Scientific Community Announce Cancer Research Program," April 3, 2001, <www.intel.com/pressroom/archive/releases/20010403corp.htm>.
43. Parallel Processing, <library.thinkquest.org/C007645/english/1-parallel.htm?tskip=1&tqtime=1020>.
44. Alternatives and forms of DC; <library.thinkquest.org/C007645/english/0-alternatives.htm?tskip=1&tqtime=0818>.
45. "Linux Clustering in Depth", *Linux Magazine*, <www.linux-mag.com/2000-10/clustering_01.html>.
46. "Distributed Computing: Power Grid", <www.zdnet.com.au/printfriendly?AT=2000048620-20269390-2>.
47. "Unraveling the Mysteries of Clustering", <www.networkcomputing.com/shared/printArticle.jhtml?article=1119/1119f2full.html>.
48. The UNIX Operating System: A Robust, Standardized Foundation for Cluster Architectures, <www.unix.org/whitepapers/cluster.html>.
49. HP 9000 Superdome Overview and Features, <www.hp.com/products/servers/scalableservers/superdome>.
50. HP High Performance XC Clusters, <www.hp.com/techservers/clusters/5982-0212EN_lo.pdf>.
51. Understanding Clustering, <otn.oracle.com/oramag/oracle/03-jul/043devejb.html>.
52. Jan Lindheim, C, "Building a Beowulf System", <www.cacr.caltech.edu/research/beowulf/tutorial/beosoft>.
53. "What's a Beowulf?" <www.phy.duke.edu/brahma/beowulf_book/node9.html>.
54. "Windows Server 2003 Clustering," <www.microsoft.com/windowsserver2003/docs/BDMTDM.doc>.
55. "Windows Server 2003 Clustering," <www.microsoft.com/windowsserver2003/docs/BDMTDM.doc>.
56. Harold, E., *JAVA Network Programming*, Sebastopol, CA: O'Reilly & Associates, Inc., 1997, pp. 26-27.
57. "Peer-to-Peer File Sharing," <banners.noticiasdot.com/termometro/boletines/docs/musica-video/varios/2002/musica_P2PWhitePaper.pdf>.
58. "P2P Search Engines," <ntrg.cs.ted.ie/undergrad/4ba2.02-03/p8.html>.
59. "GNUtella," <ntrg.cs.ted.ie/undergrad/4ba2.02-03/p5.html>.
60. "Technical: How KaZaA Works," <www.christian-gerner.de/forum/topic-1-279-279.html>.
61. Milojicic, D.; V. Kalogeraki; R. Lukose; K. Nagaraja; J. Pruyne; B. Richard; S. Rollins; and Z. Xu, "Peer-to-Peer Computing", March 2002, <www.hpl.hp.com/techreports/2002/HPI_-2002-57.pdf>.
62. "The Free Network Project," <freenet.sourceforge.net/>.
63. Oram, A., "Gnutella and Freenet Represent True Technological Innovation," <www.oreillynet.com/pub/a/network/2000/0512/magazine/gnutella.html?page=1>.
64. Oram, A., "Gnutella and Freenet Represent True Technological Innovation," <www.oreillynet.com/pub/a/network/2000/0512/magazine/gnutella.html?page=1>.
65. Gong, L., "Project JXTA: A Technology Overview," <www.jxta.org/project/www/docs/jxtaview_01nov02.pdf>.
66. AOL Instant Messenger™, <www.aim.com/index.adp>.
67. Gong, L., "Project JXTA: A Technology Overview," <www.jxta.org/project/www/docs/jxtaview_01nov02.pdf>.
68. See, S., "Project JXTA Technology Overview," <cosmos.kaist.ac.kr/~hckim/p2p/busan/jxta-overview.pdf>.
69. "Grid Computing," <searchcio.techtarget.com/sDefinition/0,sid19_gci773157,00.html>.
70. Foster, I., "Internet Computing and the Emerging Grid," <www.nature.com/nature/webmatters/grid/grid.html>.
71. "Grid Computing," <www.nr.no/dart/projects/emerging/grid.html>.
72. Casanova, H., "Distributed Computing Research Issues in Grid Computing," *ACM SIGACT News*, Vol. 33, No. 3, Sept. 2002. p. 54.
73. "Learn about SETI@home," <setiathome.ssl.berkeley.edu/learnmore.html>.
74. Hipschman, R., "How SETI@home Works," <setiathome.ssl.berkeley.edu/about_seti/about_seti_at_home_1.html>.
75. "About the Globus Alliance," <www.globus.org/about/default.asp>.
76. "Overview—The Solution: Grid Computing," <www.ud.com/solutions>.
77. Deitel, H. M. and P. J. Deitel, *Java How To Program, 5th Edition* Upper Saddle River, NJ: Prentice Hall, 2003.
78. "Introduction to JavaServer Pages™," Sun Microsystems, 2002, <developer.java.sun.com/developer/onlineTraining/webcasts/pdf/webcamp/dchen/dchen6.pdf>.
79. "The Apache Jakarta Project," <jakarta.apache.org>.
80. "The J2EE™ Tutorial—Servlet Life Cycle," <java.sun.com/j2ee/tutorial/1_3-fcs/doc/Servlets4.html>.
81. McPherson, S., "JavaServer Pages: A Developer's Perspective." April 2000, <developer.java.sun.com/developer/technicalArticles/Programming/jsp/>.
82. "Jini™ Architecture Specification," <www.sun.com/software/jini/specs/jini1.2html/jini-spec.html>.

83. "Jini™ Technology Core Platform Specification, LE-Distributed Leasing," <www.jini.org/nonav/standards/davis/doc/specs/html/lease-spec.html>.
84. "JavaSpaces Service Specification," <www.sun.com/software/jini/specs/js2_0.pdf>.
85. Freeman, E., and S. Hupfer, "Make room for JavaSpaces, Part 1," <www.javaworld.com/javaworld/jw-11-1999/jw-11-jiniology_p.html>.
86. Kreger, H., "Java Management Extensions for Application Management," <www.research.ibm.com/journal/sj/401/kreger.html>.
87. "Java Management Extensions White Paper," <java.sun.com/products/JavaManagement/wp>.
88. Deitel, H. M.; P. J. Deitel; B. DuWaldt; and L. K. Trees, *Web Services A Technical Introduction*, Upper Saddle River, NJ: Prentice Hall, 2003, p. 2.
89. Snell, J., "Web Services Interoperability," <www.xml.com/pub/a/2002/01/30/soap.html>.
90. Deitel, H. M.; P. J. Deitel; B. DuWaldt; and L. K. Trees, *Web Services A Technical Introduction*, Upper Saddle River, NJ: Prentice Hall, 2003, pp. 4-5.
91. Deitel, H. M.; P. J. Deitel; J. Gadzik; K. Lomeli; S. Santry; and S. Zhang, *Java Web Services for Experienced Programmers*, Upper Saddle River, NJ: Prentice Hall, 2003, p. 7.
92. Combs, B., "Windows NT/2000," <www.interex.org/pubcontent/enterprise/nov01/nt1101.jsp>.
93. Sun Open Net Environment (Sun ONE), <www.sun.com/software/sunone/>.
94. "Sun ONE Overview: Vision," <www.sun.com/software/sunone/overview/vision/>.
95. "Sun ONE Overview: Architecture," <www.sun.com/software/sunone/overview/architecture/index.html>.
96. Adhikari, R., "Sun Extends Web Services Strategy," *Application Development Trends*, Dec. 2001, p. 12.
97. "Sun ONE Overview: DART Model," <www.sun.com/software/sunone/overview/dart/index.html>.
98. Tanenbaum, A., and R. Renesse, "Distributed Operating Systems," *ACM Computing Surveys*, Vol. 17, No. 4, December 1985, pp. 419-470.
99. Levy, E., and A. Silberschatz, "Distributed File Systems: Concepts and Examples," *ACM Computing Surveys*, Vol. 22, No. 4, December 1990, pp. 321-374.
100. Sandberg, R., "The Sun Network Filesystem: Design, Implementation and Experience," *Proceedings of The JSENLIX Conference*, 1986, pp. 300-314, <www.cs.ucf.edu/~eurip/cop6614/sandbergnfs.pdf>.
101. Nowicki, B., "NFS: Network File System Protocol Specification," March 1989, <www.ietf.org/rfc/rfc1094.txt>.
102. Callaghan, B., "NFS Version 3 Protocol Specification," <www.ietf.org/rfc/rfc1813.txt>, June 1995.
103. Network File Systems: Version 2 Protocol Specification, <docs.freebsd.org/44doc/psd/27.nfsrhc/paper.pdf>.
104. Callaghan, B., "WebNFS Client Specification," October 1996, <www.ietf.org/rfc/rfc2054.txt>.
105. Shepler, S., "RFC 2624—NFS Version 4 Design Considerations," June 1999, <www.faqs.org/rfcs/rfc2624.html>.
106. Satyanarayanan, M., "Scalable, Secure and Highly Available Distributed File Access," *IEEE Computer*, May 1990, p. 9.
107. Morris, J. H., et al., "Andrew: A Distributed Personal Computing Environment," *Communications of the ACM*, Vol. 29, No. 3, March 1986, pp. 186-187.
108. Satyanarayanan, M., "The Evolution of Coda," *ACM Transactions on Computer Systems*, Vol. 20, No. 2, May 2002, p. 89.
109. Ousterhout, J., "A Brief Retrospective on the Sprite Network Operating System," <www.cs.berkeley.edu/projects/sprite/retrospective.html>.
110. Ousterhout, J., et al., "The Sprite Network Operating System," *IEEE Computer*, February 1988, p. 28.
111. Nelson, M., "Caching in the Sprite Network File System," *ACM Transactions on Computer Surveys*, Vol. 6, No. 1, February 1988, pp. 134-154.
112. Baker, M., and J. Ousterhout, "Availability in the Sprite Distributed File System," *Proceedings of the 4th workshop on ACM SIGOPS European workshop*, September 1990, pp. 1-4.
113. Ousterhout, J., et al., "The Sprite Network Operating System," *IEEE Computer*, February 1988, pp. 30-31.
114. Nelson, M., "Caching in the Sprite Network File System," *ACM Transactions on Computer Surveys*, Vol. 6, No. 1, February 1988, pp. 134-154.
115. "What's a Beowulf?" <www.phy.duke.edu/brama/beowulf_book/node9.html>.
116. "Windows Server 2003 Clustering," <www.microsoft.com/windowsserver2003/docs/BDMTDM.doc>.
117. "Peer-to-Peer File Sharing," <banners.noticiasdot.com/termometro/boletines/docs/musica-video/varios/2002/musica_P2PWhitePaper.pdf>.
118. "P2P Search Engines," <ntrg.cs.tcd.ie/undergrad/4ba2.02-03/p8.html>.
119. "Technical: How KaZaA Works," <www.christian-gerner.de/forum/topic-1-279-279.html>.
120. Oram, A., "Gnutella and Freenet Represent True Technological Innovation," <www.oreillynet.com/pub/a/network/2000/05/12/magazine/gnutella.html?page=1>.
121. Milojevic, D; V Kalogeraki; R. Lukose; K. Nagaraja; J. Pruyne; B. Richard; S. Rollins and Z. Xu, "Peer-to-Peer Computing", March 2002, <www.hpl.hp.com/techreports/2002/HPL-2002-57.pdf>.
122. Deitel, H. M. and P. J. Deitel, *Java How To Program, 5th Edition*, Upper Saddle River, NJ: Prentice Hall, 2003.
123. "Introduction to JavaServer Pages™," Sun Microsystems, 2002, <developer.java.sun.com/developer/onlineTraining/webcasts/pdf/webcamp/dchen/dchen6.pdf>.

864 Distributed Systems and Web Services

124. "The J2EE™ Tutorial—Servlet Life Cycle," <java.sun.com/j2ee/tutorial/1_3-fcs/doc/Servlets4.html>.
125. McPherson, S., "JavaServer Pages: A Developer's Perspective," April 2000, <developer.java.sun.com/developer/technicalArticles/Programming/jsp/>.
126. "Jini™ Architecture Specification," <www.sun.com/software/jini/specs/jini1.2html/jini-spec.html>.
127. "Jini™ Technology Core Platform Specification, LE-Distributed Leasing," <www.jini.org/nonav/standards/davis/doc/specs/html/lease-spec.html>.
128. "JavaSpaces Service Specification," <www.sun.com/software/jini/specs/js2_0.pdf>.
129. "Java Management Extensions White Paper," <java.sun.com/products/JavaManagement/wp/>.
130. Snell, X., "Web Services Interoperability," <www.xml.com/pub/a/2002/01/30/soap.html>.
131. Combs, B., "Windows NT/2000," <www.interex.org/pubcontent/enterprise/nov01/nt1101.jsp>.
132. Sun Open Net Environment (Sun ONE), <www.sun.com/software/sunone/>.
133. "Sun ONE Overview: DART Model," <www.sun.com/software/sunone/overview/dart/index.html>.
134. Deitel, H. M., P. J. Deitel, B. DuWaldt and L. K. Trees, *Web Services A Technical Introduction*, Upper Saddle River, NJ: Prentice-Hall, 2003, pp. 4-5.
135. Deitel, H. M., P. J. Deitel, J. Gadzik, K. Lomeli, S. Santry and S. Zhang, *Java Web Services for Experienced Programmers*, Upper Saddle River, NJ: Prentice Hall, 2003, p. 7.
136. Tanenbaum, A. S., and M. van Steen, *Distributed Systems Principles and Paradigms*, Upper Saddle River, NJ: Prentice Hall, 2002.
137. Foster, I. and C. Kesselman, *The Grid: Blueprint for a New Computing Infrastructure*, San Francisco, CA: Morgan Kaufmann, 1999.
138. Bester, E; I. Foster; C. Kesselman; J. Tedesco; and S. Tuecke. "GASS: A Data Movement and Access Service for Wide Area Computing Systems," *In Proceeding of the Sixth Workshop on I/O in Parallel and Distributed Systems*, 1999, pp. 78-88.

Security

*'Is it weakness of intellect, birdie?' I cried,
'Or a rather tough worm in your little inside?'*
-W. S. Gilbert-

Part 7

Computers are vulnerable to attacks that can compromise data, corrupt files and crash systems—operating system security is at the core of secure computing systems. The next chapter discusses key security topics, including cryptography (secret key and public key), authentication (biometrics, smart cards, Kerberos, single sign-on), access control, security attacks (cryptanalysis, viruses, worms, denial-of-service attacks), attack prevention (firewalls, intrusion detection systems, antivirus software), key agreement protocols, digital signatures, Public Key Infrastructure (PKI), certificates, certificate authorities, secure communication protocols (Secure Sockets Layer, Virtual Private Networks, wireless security), steganography and proprietary vs. open-source security. We include a case study on OpenBSD, arguably the world's most secure operating system.



We have forty million reasons for failure, but not a single excuse.

— Rudyard Kipling —

You can't trust code that you did not totally create yourself. (Especially code from companies that employ people like me.

—Ken Thompson—
1983 Turing Award Lecture
Association for Computing Machinery, Inc.

We have forty million reasons for failure, but not a single excuse.

—Rudyard Kipling—

There is no such thing as privacy on a computer. The view here is that if you don't want something read, don't put it on the system.

—Thomas Mandel—
quoted in *Time*, April 6, 1987

*'Is it weakness of intellect, birdie?' I cried,
'Or a rather tough worm, in your little inside?'*

- W. S. Gilbert -

Chapter 19

Security

Objectives

After reading this chapter, you should understand:

- *public-key/private-key cryptography.*
- *the role of authentication in providing secure systems.*
- *access control models, policies and mechanisms.*
- *security threats, such as viruses, worms, exploits and denial-of-service attacks.*
- *security and authentication protocols, such as SSL and Kerberos.*
- *digital signatures, digital certificates and certificate authorities.*
- *Virtual Private Networks and IPSec.*

Chapter Outline

19.1

Introduction

*Operating Systems Thinking: Ethical
Systems Design*

19.2

Cryptography

19.2.1 Secret-Key Cryptography

19.2.2 Public-Key Cryptography

*Biographical Note: Rivest, Shamir and
Adleman*

19.3

Authentication

19.3.1 Basic Authentication

19.3.2 Biometrics and Smart Cards

19.3.3 Kerberos

19.3.4 Single Sign-On

19.4

Access Control

*19.4.1 Access Rights and Protection
Domains*

19.4.2 Access Control Models and Policies

19.4.3 Access Control Mechanisms

19.5

Security Attacks

19.5.1 Cryptanalysis

19.5.2 Viruses and Worms

19.5.3 Denial-of-Service (DoS) Attacks

19.5.4 Software Exploitation

19.5.5 System Penetration

19.6

Attack Prevention and Security Solutions

19.6.1 Firewalls

*19.6.2 Intrusion Detection Systems
(IDSs)*

19.6.3 Antivirus Software

19.6.4 Security Patches

Mini Case Study: OpenBSD

Mini Case Study: Macintosh

19.6.5 Secure File Systems

19.6.6 Orange Book Security

19.7

Secure Communication

19.8

Key Agreement Protocols

19.8.1 Key Management

19.8.2 Digital Signatures

19.9

***Public-Key Infrastructure,
Certificates and Certificate
Authorities***

19.10

***Secure Communication
Protocols***

19.10.1 Secure Sockets Layer (SSL)

*19.10.2 Virtual Private Networks (VPNs)
and IP Security (IPSec)*

19.10.3 Wireless Security

19.11
Steganography

19.12
***Proprietary and Open-Source
Security***

19.13
***Case Study: UNIX Systems
Security***

Web Resources | Summary | Key Terms | Exercises | Recommended Reading | Works Cited

19.1 Introduction

As the personal computing and e-business movements expand, individuals and organizations are storing highly confidential information on their computers and transmitting sensitive information over the Internet. Consumers are submitting credit card numbers to e-commerce sites, and businesses are exposing proprietary data on the Web. At the same time, organizations are experiencing increasing numbers of security breaches. Both individuals and companies are vulnerable to data theft and attacks that can compromise data, corrupt files and crash systems. The computing industry has been responding to these needs, as organizations work to improve Internet and network security. For example, the "trustworthy computing" initiative put forth by Microsoft Chairman Bill Gates is an effort to focus the company's priorities on providing reliable, available and secure applications (see the Operating Systems Thinking feature, Ethical Systems Design).¹

Computer security addresses the issue of preventing unauthorized access to resources and information maintained by computers. Computer systems must provide mechanisms to manage security threats originating both from outside the computer (via a network connection) and from within the computer (via malicious users and software). Computer security commonly encompasses guaranteeing the privacy



Operating Systems Thinking

Ethical Systems Design

We have entrusted our lives to computers in a manner that demands ethical behavior on the part of operating systems designers.² People who work with computers and communication devices are often privy to confidential information they would not ordinarily see. Computer systems control heart pacemakers, monitor air traffic and process confidential financial, medical and criminal records. People who design computer systems must realize that these systems are used and relied upon in critical sit-

uations. Should we build systems that could fail? What is an "acceptable" level of failure? Who is responsible when computer systems fail? Should we build systems so complex that we cannot be certain they will perform properly?³ Operating systems designers need to be concerned with these and other important ethical issues.

With viruses and worms attacking our computer systems in record numbers, we must think seriously about just how vulnerable our systems are, and about

our professional responsibilities to build dependable systems. Can we continue to build systems that are so easily attacked and compromised, and employ those systems in situations where people's health and lives are at stake? It is software, not computer hardware, which has been most vulnerable to attack. Operating systems designers must conscientiously deal with the same kinds of ethical dilemmas as those faced by other professional people such as doctors, attorneys, scientists and engineers.

and integrity of sensitive data, restricting the use of computer resources and providing protection against malicious attempts to incapacitate the system. **Protection** encompasses mechanisms that shield resources such as hardware and operating system services from attack. Security is quickly becoming one of the richest and most challenging topics in computing; operating system security is at the core of a secure computing system.

This chapter also explores the fundamentals of Internet security, including the implementation of secure electronic transactions and secure networks. We discuss how to achieve network security using current technologies—including cryptography, Public Key Infrastructure (PKI), digital signatures, Secure Sockets Layer (SSL) and Virtual Private Networks (VPNs). We also examine authentication and authorization solutions, firewalls and intrusion detection systems. At the end of the chapter, we discuss UNIX security. Linux operating systems security and Windows XP operating systems security are discussed in Section 20.13 and Section 21.13, respectively.

Self Review

1. Why are security and protection important even for computers that do not contain sensitive data?
2. What is the difference between security and protection?

Ans: 1) All computers, regardless of the data they manage, are susceptible to security breaches that may crash the system, spread viruses or commit identity theft. **2)** Security addresses the issue of preventing unauthorized access to resources and information maintained by computers. Protection refers to the mechanisms that shield resources such as hardware and operating system services from attack.

19.2 Cryptography

An important goal of computing is to make massive amounts of data easily available. Electronic data transmission, especially over public wires, is inherently insecure. A solution to this problem is to make data unreadable to any unauthorized user, through encryption. Many security mechanisms rely on encryption to protect sensitive data such as passwords.

Cryptography focuses on encoding and decoding data so that it can be interpreted only by the intended recipients. Data is transformed by the use of a **cipher**, or **cryptosystem**—a mathematical algorithm for encrypting messages. A key, represented by a string of characters, is input to the cipher. The algorithm transforms the unencrypted data, or **plaintext**, into encrypted data, or **ciphertext**, using the keys as input—different keys result in different ciphertext. The aim is to render the data incomprehensible to any unintended receivers (i.e., those who do not possess the decryption key). Only the intended receivers should have the key to decrypt the ciphertext into plaintext.

First employed by the ancient Egyptians, cryptographic ciphers have been used throughout history to conceal and protect valuable information. Ancient cryp-

tographers encrypted messages by hand, usually with a method based on the alphabetic letters of the message. The ancients' two primary types of ciphers were substitution ciphers and transposition ciphers. In a **substitution cipher**, every occurrence of a given letter is replaced by a different letter. For example, if every "a" were replaced by a "b," every "b" by a "c," and so on, the word "security" would encrypt to "tfdvsjuz." The first prominent substitution cipher was credited to Julius Caesar and is referred to today as the Caesar cipher. Using the Caesar cipher, every instance of a letter is replaced by the letter in the alphabet three places to the right. For example, the word "security" would encrypt to "vhfxulwb."⁴

In a **transposition cipher**, the ordering of the letters is shifted. For example, if every other letter, starting with "s," in the word "security" creates the first word in the ciphertext and the remaining letters create the second word, the word "security" encrypts to "scrt euiy." An encrypted message becomes increasingly robust (i.e., difficult to decipher) when its cipher combines substitution and transposition ciphers. For example, using the substitution cipher in combination with the transposition cipher, the word "security" encrypts to "tdsu fvjz."⁵

The greatest weakness of many early ciphers was their reliance on the sender and receiver remembering the encryption algorithm and maintaining its secrecy. Such algorithms are called **restricted algorithms**. Restricted algorithms are not feasible to implement for a large group of people. Imagine if the security of U.S. government communications relied on every employee keeping a secret—the encryption algorithm could easily be compromised.

Modern cryptosystems rely on algorithms that operate on the individual bits or blocks (a group of bits) of data, rather than letters of the alphabet. Encryption and decryption keys are binary strings with a given key length. For example, 128-bit encryption systems have a key length of 128 bits. Longer keys have stronger encryption; it takes more time and computing power to "break" the encryption. Longer keys also require more processing time to encrypt and decrypt data, reducing system performance. The growth of computer networking has made the security/performance trade-off more challenging.

Until January 2000, the U.S. government placed restrictions on the strength of cryptosystems that could be exported from the United States by limiting the key length of exported encryption algorithms. Today, the regulations are less stringent. Any cryptography product may be exported, as long as the end user is not a foreign government or a member of a country with embargo restrictions.⁶

Self Review

1. Consider a cipher that randomly reorders the letters of each word in a message. Why is such a cipher inappropriate for encryption?
2. What is the primary weakness of restricted algorithms?

Ans: 1) The receiver would be unable to decipher the ciphertext because it would have no way of knowing how the letters were reordered. 2) The sender and receiver must remember the encryption algorithm and maintain its secrecy; thus, restricted algorithms are not feasible to implement for a large group of people.

19.2.1 Secret-Key Cryptography

Symmetric cryptography, also known as **secret-key cryptography**, uses the same secret key to encrypt and decrypt a message (Fig. 19.1). In this case, the sender encrypts a message using the secret key, then sends the encrypted message to the intended recipient, who decrypts the message using the same secret key. A limitation of secret-key cryptography is that before two parties can communicate securely, they must find a secure way to exchange the secret key.

A solution is to have the key delivered by a courier or other mail service. While this approach may be feasible when two parties communicate, it does not scale well to large networks. Moreover, secret-key encryption cannot be considered completely secure, because message privacy and integrity can be compromised if the key is intercepted as it is passed between the sender and the receiver. Also, because both parties in the transaction use the same key to encrypt and decrypt a message, one cannot determine which party created a message. This enables a third party, posing as one of the two authorized parties, to create a message after capturing the key. Finally, to maintain private communications, a sender needs a different secret key for each receiver. Consequently, secure computing in large organizations would necessitate maintaining large numbers of secret keys for each user, requiring significant data storage.

An alternative approach to the key-exchange problem is to create a central authority, called a **key distribution center (KDC)**, which shares a different secret key with every user in the network. The key distribution center generates a **session** key to be used for a transaction (Fig. 19.2), then delivers the session key to the sender and receiver, encrypted with the secret key they each share with the key distribution center.

For example, suppose a merchant and a customer wish to conduct a secure transaction. They each share unique secret keys with the key distribution center. The KDC generates a session key for the merchant and customer to use in the

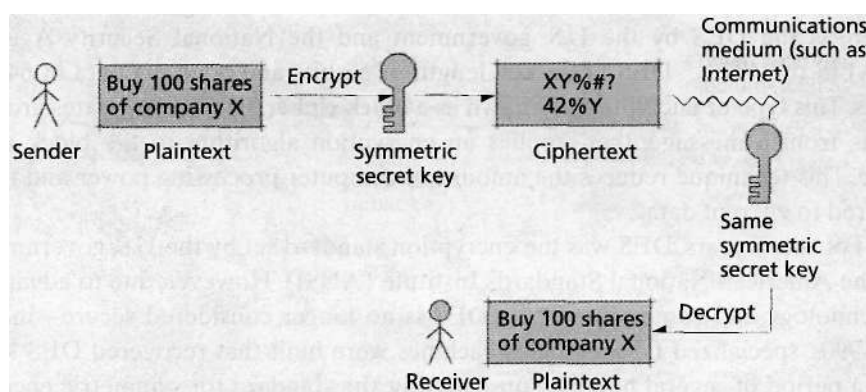


Figure 19.1 | Encrypting and decrypting a message using a secret key.

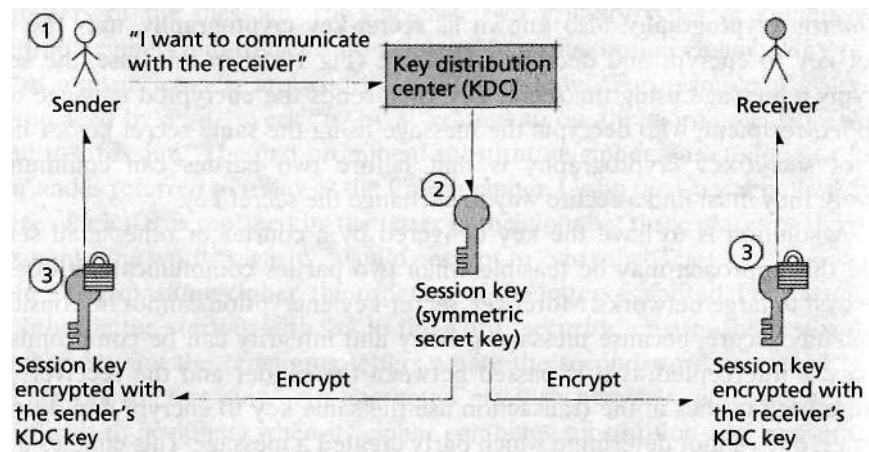


Figure 19.2 | Distributing a session key with a key distribution center.

transaction, then sends the session key for the transaction to the merchant, encrypted using the secret key the merchant already shares with the center. The KDC sends the same session key for the transaction to the customer, encrypted using the secret key the customer already shares with the KDC. Once the merchant and the customer obtain the session key for the transaction, they can communicate with each other, encrypting their messages using the shared session key.

A key distribution center reduces the number of courier deliveries of secret keys to each user in the network. In addition, it gives users a new secret key for each communication with other users in the network, which greatly increases the level of security of the network. However, if the security of the key distribution center is compromised, then the security of the entire network is also compromised.

One commonly used symmetric encryption algorithm is the **Data Encryption Standard (DES)**. Horst Feistel of IBM created the **Lucifer algorithm**, which was chosen as the DES by the U.S. government and the National Security Agency (NSA) in the 1970s.⁷ DES has a key length of 56 bits and encrypts data in 64-bit blocks. This type of encryption is known as a **block cipher**, because it creates groups of bits from a message, then applies an encryption algorithm to the block as a whole. This technique reduces the amount of computer processing power and time required to encrypt data.

For many years, DES was the encryption standard set by the U.S. government and the American National Standards Institute (ANSI). However, due to advances in technology and computing speed, DES is no longer considered secure—in the late 1990s, specialized DES cracker machines were built that recovered DES keys after a period of several hours.⁸ Consequently, the standard for symmetric encryption was replaced by **Triple DES**, or **3DES**, a variant of DES that is essentially three DES systems in series, each with a different secret key that operates on a block.

Though 3DES is more secure, the three passes through the DES algorithm increase encryption overhead, leading to reduced performance.

In October 2000, the U.S. government selected a more secure standard for symmetric encryption to replace DES, called the **Advanced Encryption Standard (AES)**. The **National Institute of Standards and Technology (NIST)**—which sets the cryptographic standards for the U.S. government—chose **Rijndael** as the encryption method for AES. Rijndael is a block cipher developed by Dr. Joan Daemen and Dr. Vincent Rijmen of Belgium. Rijndael can be used with key sizes and block sizes of 128, 192 or 256 bits. Rijndael was chosen as the AES over four other finalists due to its high level of security, performance, efficiency, and flexibility and its low memory requirement for computing systems.⁹

Self Review

1. Discuss the advantages and disadvantages of secret-key cryptography.
2. What limits the strength of most encryption algorithms?

Ans: 1) An advantage is that encryption and decryption via secret-key cryptography are straightforward because both parties share the same key, making secret-key encryption highly efficient. Disadvantages are that it does not provide authentication and it requires a secure way to transmit the secret key so that a third party does not intercept it. **2)** The primary limitation to most encryption algorithms is the amount of computational power required to implement them.

19.2.2 Public-Key Cryptography

In 1976, Whitfield Diffie and Martin Hellman, researchers at Stanford University, developed **public-key cryptography** to solve the problem of securely exchanging symmetric keys. Public-key cryptography is asymmetric in that it employs two inversely related keys: a **public key** and a **private key**. The private key is kept secret by its owner and the public key is freely distributed. If the public key encrypts a message, only the corresponding private key can decrypt it (Fig. 19.3) and vice

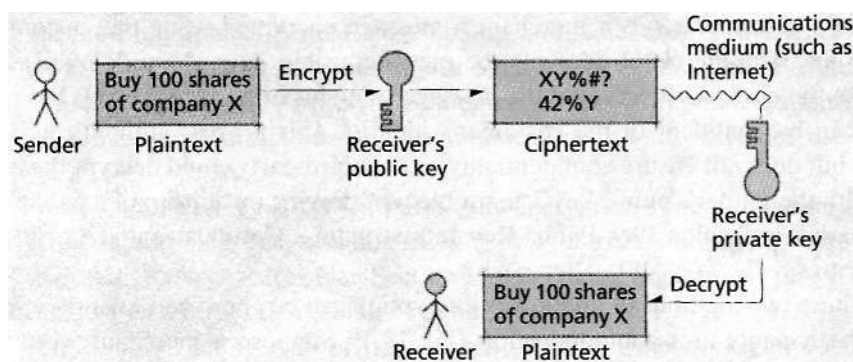


Figure 19.7 | Encrypting and decrypting a message using public-key cryptography.

versa. Each party in a transaction possesses both a public key and a private key. To transmit a message securely, the sender uses the receiver's public key to encrypt the message. The receiver then decrypts the message, using his or her unique private key. Assuming that the private key has been kept secret, the message cannot be read by anyone other than the intended receiver. Thus the system ensures the privacy of the message.

Secure public-key algorithms depend on a private key that is computationally infeasible to deduce from the public key within a reasonable amount of time. Such algorithms are called "one-way" or "trap-door" functions, because the message requires little computation to encrypt using the public key, but takes significant computing power and time to decrypt without knowledge of the private key. Moderately secure encryption requires years for a single computer to decrypt. [Note: Distributed.net's RC5 competition offers a cash award to decrypt a message encrypted by a one-way function. The competition attracts thousands of users that work in parallel to discover the key. Recently, over 300,000 participants devoted computer power to crack a message encrypted using a 64-bit key. The key was found in August of 2002 after 1,757 days of computation, equivalent to 46,000 2GHz processors working in parallel.¹⁰ The most powerful computer at the time was the Earth Simulator, which contained 5,120 500MHz processors.]

The security of the system relies on the secrecy of the private keys. Therefore, if a third party obtains the private key used in decryption, the security of the entire system is compromised. If system integrity is compromised, the user can simply change the key, rather than change the entire encryption or decryption algorithm.

Both the public key or the private key can be used to encrypt or decrypt a message. For example, if a customer uses a merchant's public key to encrypt a message, only the merchant can decrypt the message, using the merchant's private key. Thus, the merchant's identity can be authenticated, because only the merchant knows the private key. However, the merchant cannot validate the customer's identity, because the encryption key the customer used is publicly available.

If the decryption key is the sender's public key and the encryption key is the sender's private key, the sender of the message can be authenticated. For example, suppose a customer sends a merchant a message encrypted using the customer's private key. The merchant decrypts the message using the customer's public key. Because the customer encrypted the message using his or her private key, the merchant can be confident of the customer's identity. This process authenticates the sender but does not ensure confidentiality, as any third party could decrypt the message with the sender's public key. The problem of proving ownership of a public key is discussed in Section 19.9, Public-Key Infrastructure, Certificates and Certificate Authorities.

These two methods of public-key encryption can be combined to authenticate both participants in a communication (Fig. 19.4). Suppose a merchant wishes to send a message securely to a customer so that only the customer can read it, and also to provide proof to the customer that the merchant sent the message. First, the

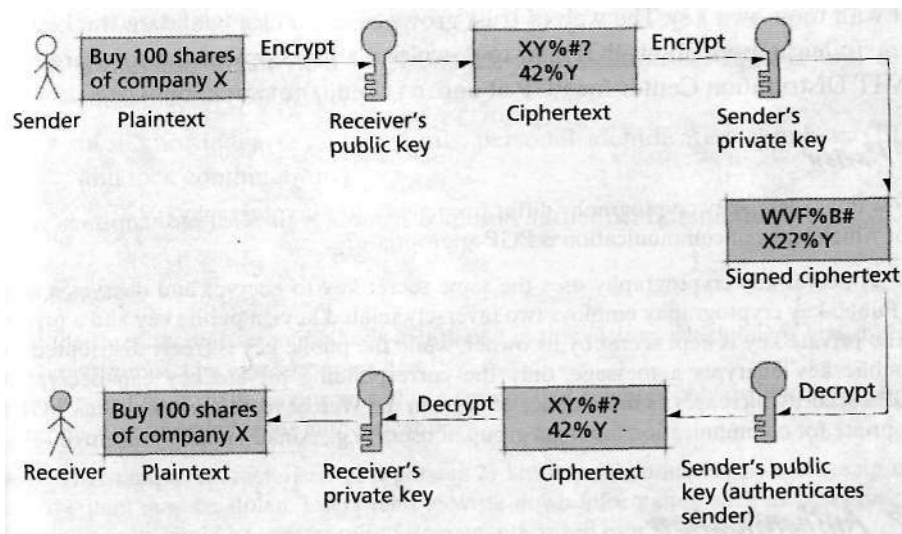


Figure 19.4 | Authentication with a public-key algorithm.

merchant encrypts the message using the customer's public key. This step guarantees that only the customer can read the message. Then the merchant encrypts the result using the merchant's private key, which proves the identity of the merchant. The customer decrypts the message in reverse order. First, the customer uses the merchant's public key. Because only the merchant could have encrypted the message with the inversely related private key, this step authenticates the merchant. Then the customer uses the customer's private key to decrypt the next level of encryption. This step ensures that the content of the message remained private in the transmission, because only the customer has the key to decrypt the message.

The most commonly used public-key algorithm is **RSA**, an encryption system developed in 1977 by MIT professors Ron Rivest, Adi Shamir and Leonard Adleman (see the Biographical Note, Rivest, Shamir and Adleman).¹¹ Their encryption products are built into hundreds of millions of copies of the most popular Internet applications, including Web browsers, commerce servers and e-mail systems. Most secure e-commerce transactions and communications on the Internet use RSA products.

Pretty Good Privacy (PGP), a public-key encryption system that encrypts e-mail messages and files, was designed in 1991 by Phillip Zimmermann.¹² PGP can also provide digital signatures (see Section 19.8.2, Digital Signatures) that confirm the identity of an author of an e-mail or public posting.

PGP is based on a "web of trust"; each client in a network can vouch for another client's identity to prove ownership of a public key. The "web of trust" is used to authenticate each client. If users know the identity of a public key holder,

through personal contact or another secure method, they validate the key by signing it with their own key. The web of trust grows as more users validate the keys of others. To learn more about PGP and to download a free copy of the software, visit the MIT Distribution Center for PGP at web.mit.edu/network/pgp.html.

Self Review

1. How does secret-key cryptography differ from public-key cryptography?
2. For which types of communication is PGP appropriate?

Ans: 1) Secret-key cryptography uses the same secret key to encrypt and decrypt a message. Public-key cryptography employs two inversely related keys: a public key and a private key. The private key is kept secret by its owner, while the public key is freely distributed—if the public key encrypts a message, only the corresponding private key can decrypt it. 2) PGP's security increases as the number of users in the Web of trust increases. Thus, PGP is appropriate for communication among a group of users (e.g., e-mail or public forums).

19.3 Authentication

Identifying users and the actions they are allowed to perform is vital to maintaining a secure computing system. A user can be identified by:



Biographical Note

Rivest, Shamir and Adleman

Ronald Rivest, Adi Shamir and Leonard Adleman are the inventors of the RSA cryptography algorithm. This algorithm is used in over one billion copies of secure software applications.¹³

Rivest, Shamir and Adleman met in the 1970s while working as assistant professors in Math at MIT. When Whitfield Diffie and Martin Hellman published their new theory of public-key cryptography in 1977, Rivest and Shamir tried to find a practical implemen-

tation. They enlisted Adleman to test out their ideas; he disproved over 40 of their designs until they finally found the mechanism that they published as "A Method for Obtaining Digital Signatures and Public-Key Cryptosystems" in *Communications of the ACM* in 1978.¹⁴ Rivest, Shamir and Adleman obtained a patent for their algorithm and started the RSA Data Security company (now named RSA Security) in 1982.^{15, 16}

Together they won the 2002 Turing Award for their development of the RSA code.¹⁷

The RSA Security company uses the RSA algorithm in security software including authentication mechanisms, smart cards, digital certificates and software development kits (SDKs) to facilitate the development of secure applications. RSA Security dominates the field of authentication software, controlling over 65 percent of the market.¹⁸

- a unique characteristic of the person (e.g., fingerprints, voiceprints, retina scans and signatures).
- ownership of an item (e.g., badges, identification cards, keys and smart cards).
- user knowledge (e.g., passwords, personal identification numbers (PINs) and lock combinations).

In the sections that follow, we discuss common authentication methods.

Self Review

1. Of the three identification methods mentioned in this section, which one is least likely to be compromised by eavesdroppers?
2. How might security be compromised in a system that requires only ownership of an item or only user knowledge for authentication?

Ans: 1) A unique characteristic of a person. 2) Under authentication by ownership of an item, the item may be stolen. Users tend to write down information such as passwords and PINs, so security could be compromised if an unauthorized user finds such a record.

19.3.1 Basic Authentication

The most common authentication scheme is simple **password protection**.¹⁹ The user chooses a password, memorizes it and then enters it into the system to gain admission to a resource or system. Most systems suppress the display of a password on the screen by replacing password text with masking characters (which display "dummy" characters—often asterisks—for password characters entered) or by hiding password input.

Password protection introduces several weaknesses to a secure system.²⁰ Users tend to choose passwords that are easy to remember, such as the name of a spouse or pet. Someone who has obtained personal information about the user might try to log in several times using passwords that are characteristic of the user; several repeated attempts might result in a security breach. Some early systems limited users to short passwords; these systems were easily compromised by simply attempting all possible passwords—a technique known as **brute-force cracking**.

Most systems today require longer passwords that include both alphabetic and numeric characters to thwart such penetration attempts. Some systems even prohibit use of dictionary words as password values. However, long passwords do not necessarily improve the security of a system; if passwords are difficult to remember, users will be more inclined to make a note of them, making it easier for an intruder to obtain a password.

Penetration of an operating system's defenses need not necessarily result in a significant security compromise. For example, suppose an intruder manages to gain access to a system's master list of passwords. If the password file were stored in plaintext, such penetration would enable the intruder to access any information on the system by assuming the identity of any user. To reduce the effectiveness of a stolen password file, many operating systems encrypt the password file or store only

hash values for each password. In this case, a copy of the password file is of little use unless the intruder can decrypt the passwords. To further thwart attempts at obtaining passwords, several systems employ **password salting**, which is a technique that inserts characters at various positions in the password before encryption (Fig. 19.5). Note how, in Fig. 19.5, a small amount of salting can significantly alter ciphertext, even when using a weak cipher such as a substitution cipher (base 64 encoding). Salting can prevent intruders from obtaining an encryption key based on patterns produced by common passwords after encryption.

Users are encouraged to change their passwords often; even if an intruder obtains a password, it may be changed before any real damage can be done. Some systems require that users choose new passwords on a regular basis. Unfortunately, some users will reuse two or three passwords cyclically, reducing overall security. In response, several authentication systems prohibit repeated use of a user's most recent password choices.

A simple defense against brute-force cracking and repeated password trials is to limit the number of login attempts that may be initiated in any period of time from a single terminal or workstation (or from a single account). Certainly, people make typing errors when attempting to log in, but it is unreasonable for someone who knows the correct password to require tens, hundreds or thousands of tries. Therefore, a system might allow three or four tries, then disconnect the terminal for several seconds or minutes. After a waiting period, the terminal may be reconnected.

SelfReview

- 1. (T/F) Longer and more complicated passwords guarantee higher security.
- 2. How does salting improve password security?

Ans: 1) False. Longer and more complicated passwords usually result in the user writing down the password, enabling intruders to steal it. 2) Salting can prevent intruders from determining an encryption key based on patterns produced by common passwords after encryption.

19.3.2 Biometrics and Smart Cards

An innovation in security, once limited to the movies but becoming increasingly common in today's secure systems, is **biometrics**. Biometrics uses unique personal information—such as fingerprints, eyeball iris scans or face scans—to identify a user. The number of passwords an average user must remember has increased due to the prolif-

<i>Plaintext</i>	<i>Ciphertext</i>
password	cGFzc3dvcmQ=
psasaswlord	cHNhc2Fzd2xvcnRk
newpassword	bmV3cCFzc3dvcmQ=
nsewaplatssewodrd	b nM d2FwbGF0c3N1d29kcmQ=

Figure 19.5 | Salting passwords (Base 64 encoding).

eration of sensitive data transmitted over unsecure channels. [Note: People in the security community prefer unsecure to insecure.] As a result, passwords have become an increasing burden to secure computing. This trend can be detrimental to security, as users record passwords on paper or employ the same password for several accounts. Consequently biometrics has become an attractive alternative to passwords, and the cost of biometric devices has dropped significantly.

Fingerprint-scanning, face-scanning and iris-scanning devices are replacing the use of passwords to log into systems, check e-mail or access secure information over a network. Each user's fingerprint, face scan or iris scan is stored in a secure database. Each time a user logs in, his or her scan is compared with the database. If a match is made, the login is successful. Two companies that specialize in biometric devices are IriScan (www.iriscan.com) and Keytronic (www.keytronic.com). For additional resources, see the Web Resources section at the end of the chapter.

Although passwords are currently the predominant means of authentication in computer systems and e-commerce, several platforms have embraced biometrics. In 2000, Microsoft announced its Biometric Application Programming Interface (BAPI), which is included in its Windows 2000 and Windows XP operating systems to simplify the integration of biometrics into personal and corporate systems.²¹

Keyware Inc. (www.keyware.com) has implemented a wireless biometrics system that stores user voiceprints on a central server. Keyware also created **layered biometric verification (LBV)**, which uses multiple physical measurements—face, finger and voice prints—simultaneously. The LBV feature enables a wireless biometrics system to combine biometrics with other authentication methods, such as personal identification numbers (PIN) and PKI (Public Key Infrastructure; see Section 19.9, Public-Key Infrastructure, Certificates and Certificate Authorities).²²

Identix, Inc. (www.identix.com) also provides biometrics authentication technology for wireless transactions. The Identix fingerprint-scanning device is embedded in handheld devices. The Identix service offers transaction management and content protection services. Transaction management services prove that transactions took place, and content protection services control access to electronic documents, including limiting a user's ability to download or copy documents.²³

A **smart card**, often designed to resemble a credit card, is typically used to perform authentication and store data. The most popular smart cards are memory cards and microprocessor cards. Memory cards can store and transfer data. Microprocessor cards contain computer components, managed by an operating system, that provide security and storage. Smart cards are also characterized by their interface to reading devices. One is a contact interface, whereby smart cards are inserted into a reading device, requiring physical contact between the device and the card for data transfer. Alternatively, a contactless interface allows data to be transferred without physical contact between the reader and the card, often accomplished with an embedded wireless device in the card.²⁴

Smart cards can store private keys, digital certificates and other information necessary for implementing PKI. They may also store credit card numbers, personal

contact information, and the like. Each smart card can be used in combination with a PIN. This feature provides two levels of security by requiring the user to possess a smart card and know the corresponding PIN to access information stored on the card. To further strengthen security, some microprocessor cards will delete or corrupt stored data in the event of malicious tampering with the card. Smart-card PKI allows users to access information from multiple devices using the same smart card.

Two-factor authentication employs two means to authenticate the user, such as biometrics or a smart card used in combination with a password. Though this system could potentially be compromised, using two methods of authentication typically provides better security than passwords alone.

Self Review

1. Why is it difficult for an unauthorized user to gain access in a system that uses biometrics for security?
2. Explain one disadvantage to storing unencrypted user information on a smart card.

Ans: 1) It is difficult to replicate another user's physical features. 2) If the smart card is stolen, another individual could access that user's potentially sensitive information.

19.3.3 Kerberos

Internal computer attacks (i.e., attacks that originate from a valid user) are common and can be extremely damaging. For example, disgruntled employees with network access can disable an organization's network or steal valuable proprietary information. It is estimated that 70-90 percent of attacks on corporate networks are internal.²⁵ A centralized, secure authentication system can facilitate a fast response to such security attacks. **Kerberos**, a freely available, open-source protocol developed at MIT, can provide protection against internal security attacks. It employs secret-key cryptography to authenticate users in a network and to maintain the integrity and privacy of network communications.^{26,27}

Authentication in Kerberos is handled by an authentication server and a secondary **Ticket Granting Service (TGS)**. This system is similar to the key distribution centers described in Section 19.2.1, Secret-Key Cryptography. The authentication server authenticates a client's identity to the TGS; the TGS authenticates the client's rights to access specific network services.

Each client in the network shares a secret key with the Kerberos system. This secret key may be stored by multiple TGSs in the Kerberos system. The Kerberos system works as follows:²⁸

1. The client begins by submitting a username and password to the Kerberos authentication server.
2. The authentication server maintains a database of all clients in the network. If the username and password are valid, the authentication server returns a **Ticket-Granting Ticket (TGT)** encrypted with the client's secret key. Because the secret key is known only by the authentication server and

the client, only the client can decrypt the TGT, thus authenticating the client's identity.

3. The client sends the decrypted TGT to the Ticket Granting Service to request a **service ticket**. The service ticket authorizes the client's access to specific network services. Service tickets are assigned an expiration time and can be renewed or revoked by the TGS. If the TGT is valid, the TGS issues a service ticket encrypted with the client's session key.
4. The client then decrypts the service ticket, which it then presents to gain access to networked resources.

Self Review

1. In Kerberos authentication, why must the connection between clients and the authentication server be secure?
2. Why is it appropriate for tickets to expire?

Ans: 1) If the connection is not secure, eavesdroppers could steal user names and passwords to gain access to secure resources. 2) It is appropriate for tickets to expire because it forces users to reauthenticate often. For example, if an authorized user leaves a terminal while logged in, the ticket can expire before an unauthorized user can cause significant damage.

19.3.4 Single Sign-On

Single sign-on systems simplify the authentication process by allowing the user to log in once, using a single password. Users authenticated via a single sign-on system can then access multiple applications across multiple computers. Sign-on passwords should be closely guarded, because if a password becomes available to attackers, all applications protected by that password can be accessed and attacked.

Workstation login scripts are the simplest form of single sign-on. Users log in at their workstations, then choose applications from a menu. The workstation login script sends the user's password to the application servers to authenticate the user for future access to those applications. Many workstation login scripts do not provide a sufficient level of security, because user passwords are often stored on the client computer in plaintext. Even if the script uses simple password encryption, that algorithm must be present on the system, meaning that any intruder that gains access to the computer will be able to compromise the encryption.

Authentication server scripts authenticate users via a central server. The central server controls connections between the user and the applications the user wishes to access. Authentication server scripts are more secure than workstation login scripts because passwords are maintained on the server, which is generally more secure than the client computer. However, if the security of the server is compromised, the security of the entire system is also compromised.

The most advanced single sign-on systems employ **token-based authentication**. Once a user is authenticated, a unique **token** is issued to enable the user to access specific applications. The login that creates the token is secured by encryption or a single password. For example, Kerberos uses token-based authentication,

in which the service ticket acts as the token. A key problem with token authentication is that applications must be modified to accept tokens, rather than traditional login passwords.²⁹

Currently, the three leaders in development of single sign-on technology are the Liberty Alliance Project (www.projectliberty.org), Microsoft and Novell. The Liberty Alliance Project is a consortium of technology and security organizations working to create an open single sign-on solution. Microsoft's .NET Passport and Novell's SecureLogin are also viable solutions, though they are proprietary. To protect the privacy of information submitted to single sign-on and other applications, the **Platform for Privacy Preferences (P3P)** allows users to control the personal information that sites collect.^{30, 31}

Self Review

1. Of the three single sign-on services, which is the most secure?
2. In what way are workstation login scripts more secure than authentication server scripts?

Ans: 1) Token-based authentication is the most secure. However, it is also the most complicated scheme. 2) Workstation login scripts are more secure in that the security for all of the other users in the system is not compromised if the security of a workstation login script is compromised.

19.4 Access Control

As a resource manager, the operating system must carefully guard against unintentional and malicious use of computer resources. Consequently, today's operating systems are designed to protect operating system services and sensitive information from users and/or software that have gained access to computer resources. Access rights protect system resources and services from potentially dangerous users by restricting or limiting the actions that can be performed on the resource. These rights are typically managed by access control lists or capability lists.

19.4.1 Access Rights and Protection Domains

The key to operating system security is to control access to internal data and resources. **Access rights** define how various subjects can access various objects. Subjects may be users, processes, programs or other entities. Objects are resources such as hardware, software and data; they may be physical objects that correspond to disks, processors or main memory. They may also be abstract objects that correspond to data structures, processes or services. Subjects can also be objects of the system; one subject may have rights to access another. Subjects are active entities, objects are passive. As a system runs, its population of subjects and objects tends to change. The manner in which a subject can access an object is called a **privilege** and can include reading, writing and printing.

Objects must be protected from subjects. If a process were allowed to access every resource on a system, a user could maliciously (or inadvertently) compromise

system security or cause other programs to crash. To prevent such events from occurring, each subject must obtain authorization to access objects within a system.

A **protection domain** is a collection of access rights. Each access right in a protection domain is represented as an ordered pair with fields for the object name and its corresponding privileges. A protection domain is unique to a subject. For example, if a user can read and write the file `example.txt`, the corresponding ordered pair for this user's access right can be represented by `<example.txt, { read, write }>`.³²

The most common access rights are read, write and execute. Some subjects may also grant access rights to other subjects. In most computing systems, the administrator possesses all access rights and is responsible for managing other users' rights.

Access rights may be copied, transferred or propagated from one domain to another. Copying an access right simply entails granting a right of one user to another user. When an access right is transferred from subject A to subject B, subject A's access right is revoked upon completion of the transfer. Propagating an access right is similar to copying an access right; however, in addition to sharing the original access right, both subjects can also copy the right to other subjects.

When a subject no longer needs access to an object, access rights can be revoked. Several issues arise—should revocation be immediate or delayed? Should revocation apply to all objects or a select few? Should revocation apply to specific subjects or an entire domain? Should revocation be permanent or temporary? Each implementation of access rights management addresses revocation differently; we discuss several implementations in Section 19.4.3.³³

Self Review

1. (T/F) The term *subject* always refers to users in a system.
2. Explain the difference between copying access rights and propagating access rights.

Ans: 1) False. The term *subject* can refer to users, processes, programs and other entities.
 2) If subject A copies a right to subject B, subject B will not be able to grant that right to other subjects. However, if subject A propagates a right to subject B, subject B will be able to grant that right to other subjects.

19.4.2 Access Control Models and Policies

Access control can be divided into three conceptual levels: models, policies and mechanisms. A **security model** defines a system's subjects, objects and privileges. A **security policy**, which is typically specified by the user and/or system administrator, defines which privileges to objects are assigned to subjects. The **security mechanism** is the method by which the system implements the security policy. In many systems, the policy changes over time as the system's set of resources and users change, but the security model and mechanisms that implement access control do not require modification, so the security policy is separated from the mechanism and model.

A popular security model organizes users into classes, as discussed in Section 13.8.2, Access Control by User Classes. A drawback is that access rights are

stored in each file and specify a single owner and group, so at most one group can access a particular file. Further, the system might need to modify group permissions for several files when assigning new access rights to a group of users—an error-prone and time-consuming process.

In the **role-based access control (RBAC)** model, users are assigned **roles**, each typically representing a set of tasks assigned to a member of an organization. Each role is assigned a set of privileges, which define the objects that users in each role can access.³⁴ Users can belong to multiple roles; administrators need only modify permissions for a single role to change access rights for a group of users. The appeal of RBAC is that it assigns meaningful relationships between subjects and objects that are not limited by classes such as owners and groups.

For example, consider an academic computer system with the following roles: faculty members create and grade assignments, students submit completed assignments and staff members post grades to student transcripts. Under the RBAC model, this system consists of three roles (students, faculty and staff), two objects (assignments and grades) and three permissions (read, modify and create). In this example, faculty have permission to create, read and modify both assignments and grades; students have permission to read and modify copies of coursework and read grades; staff have permission to read and modify grades.

Although security policies vary to meet the needs of users in a system, most policies incorporate the principle of least privilege—a subject is granted access only to the objects it requires to perform its tasks. Policies can also implement discretionary or mandatory access control depending on the security needs of the environment. Most UNIX-based systems follow the **discretionary access control (DAC)** model, whereby the creator of an object controls the permissions for that object. On the contrary, **mandatory access control (MAC)** policies predefine a central permission scheme by which all subjects and objects are controlled. MAC is found in many high-security installations, such as classified government systems.³⁵

Self Review

1. Why are access control policies and mechanisms typically separated?
2. How might MAC be more secure than DAC?

Ans: 1) Policies often change without requiring modification to their underlying implementation; changing policies requires less work if the security policy is separated from the security mechanism. 2) MAC prevents users from accidentally or intentionally assigning permissions that might compromise system security (e.g., the owner of a file assigning public access to sensitive information).

19.4.3 Access Control Mechanisms

In this section we discuss various techniques that an operating system can employ to manage access rights. Access control matrices match subjects and objects to the appropriate access rights. The concept behind the model is simple; however, most

systems contain many subjects and objects, resulting in a large matrix that is an inefficient means for access control. Access control lists and capability lists are derived from the principle of least privilege and are often more efficient and flexible methods of managing access rights. Section 20.13.2, Access Control Methods, and Section 21.13.2, Authorization, discuss how Linux and Windows XP, respectively, use access control methods to secure their systems.

Access Control Matrices

One way to manage access rights is in an access **control matrix**. The various subjects are listed in the rows, and the objects to which they require access are listed in the columns. Each cell in the matrix specifies the actions that a subject (defined by the row) can perform on an object (defined by the column). Access rights in an access control matrix are granted on the basis of least privilege, so if an access right is not explicitly described in the matrix, the user has no access rights to the object.³⁶

Because an access control matrix places all permission information in a central location, the matrix should be one of the most closely guarded entities in an operating system. If the matrix is compromised, any resources that were protected by the access rights defined in the matrix are susceptible to attack as well.

The access control matrix in Fig. 19.6 represents the access rights for the users (Alice, Bob, Chris, David and Guest) to the objects (File A, File B and Printer). The privileges that a user can obtain for an object are read, write and print. The read and write access rights apply only to files on the system, in this case, File A and File B. In some environments, not every user has access to the printer—a user must explicitly have the print privilege to send content to the printer. Any access right that is followed by an asterisk (*) can be copied from one user to another. In this access control matrix, Alice has all access rights as well as the ability to assign these rights to other users. David cannot access File A because there is no entry in the corresponding cell in the matrix. The Guest account contains no access rights by default. A Guest can access resources only when the right is explicitly granted by another user. Although generating and interpreting an access control matrix is straightforward, the matrix can become large and sparsely populated.

	File A	File B	Printer
Alice	Read* Write*	Read* Write*	Print*
Bob	Read* Write	Read* Write	Print
Chris	Read		Print
David		Read	
Guest			

Figure 19.6 | Access control matrix for a small group of subjects and objects.

Access Control Lists

An access control list stores the same data as an access control matrix, but it maintains a record of only those entries that specify an access right. The access control list for a system can be based on the rows of a matrix (the subjects) or the columns (the objects). For each object in an operating system, an access control list contains entries for each subject and the privileges associated with that subject for the object. When a subject attempts to access an object, the system searches the access control list for that object to find any privileges for the subject.³⁷

A drawback to this method is the inefficiency with which the operating system determines user privileges for a particular object. The access control list for each object contains an entry for every subject with privileges for that object—a potentially large list. Each time an object is accessed, the system must search the list of subjects to find the proper privileges. It is difficult to determine which access rights belong to a certain protection domain when using access control lists; the access list for every object in the system must be searched for entries regarding that particular subject.

The access control list in Fig. 19.7 represents the set of access rights that were established in the access control matrix of Fig. 19.6. The implementation is smaller because empty entries in the matrix are not present. If an object does not contain an entry for a particular user, that user has no privileges for the object.

Capability Lists

A capability is a pointer or token that grants privileges to a subject that possesses it. It is analogous to a ticket used to gain access to a sporting event. Capabilities ordinarily are not modified, but they can be reproduced. Recall that a protection domain defines the set of privileges between subjects and objects. Alternatively, one can define the protection domain as the set of capabilities belonging to a subject.³⁸

A capability is often implemented as a unique object identifier. Capabilities are granted to a subject, which presents the token for all subsequent accesses to the

```

1  File A:
2    <Alice, {read*, write*}>
3    <Bob, {read*, write}>
4    <Chris, {read}>
5  File B:
6    <Alice, {read*, write*}>
7    <Bob, {read*, write}>
8    <David, {read}>
9  Printer:
10   <Alice, {print*}>
11   <Bob, {print}>
12   <Chris, {print}>

```

Figure 19.7 | Access control list derived from the access control matrix.

object. Capabilities are created by carefully guarded operating system routines. A subject that possesses a capability may perform certain operations, including creating copies of the capability or passing it as a parameter.

Upon creation of an object, a capability for that object is created. This original capability includes full privileges for the new object. The subject that creates the object may pass copies of the capability to other subjects. Likewise, a subject receiving a capability may use it to access the object, or the subject may create additional copies and pass them to other subjects. When one subject passes a capability to another, it may reduce the associated privileges. Thus, as a capability propagates through the system, its privilege set can either remain the same or decrease in size.

Users must be prevented from creating capabilities arbitrarily. This can be accomplished by storing capabilities in segments that user processes cannot access.

The identifier in a capability may be implemented as a pointer to the desired object, or it may be a unique bit sequence (i.e., a token). Pointers simplify access to the address at which the object is stored, but all such pointers in the system must be updated if the object is moved, which can degrade performance. When using tokens, capabilities do not rely on the object's location in memory. However, because a token does not specify the location of its corresponding object, tokens require that the object's address be determined when the capability is first used. A hashing mechanism implements token-based capabilities efficiently; high-speed caches often reduce the overhead of repeated references to the same object.

Systems that employ capabilities can suffer from the "lost object" problem. If the last remaining capability for an object is destroyed, the associated object can no longer be accessed. To prevent this problem, many operating systems ensure that the system always maintains at least one capability for each object.

Controlling the propagation of capabilities is a difficult problem. Systems generally do not allow direct manipulation of capabilities by users; instead capability manipulation is performed by the operating system on behalf of the users. Keeping track of capabilities is an important task that becomes difficult in multiuser systems containing a large numbers of capabilities. Many systems employ a directory structure to manage their capabilities.³⁹

Self Review

1. In what type of environment are access control lists more appropriate than access control matrices?
2. Discuss the advantages and disadvantages of token-based and pointer-based capabilities.

Ans: 1) Access control lists are more appropriate than access control matrices when the matrices are sparse. In this case, access control lists require much less space than access control matrices. **2)** Pointers simplify access to the address at which the object is stored, but all such pointers in the system must be updated if the object is moved, which can degrade performance. When using tokens, capabilities are independent of the object's location in memory. However, because a token does not specify the location of its corresponding object, tokens require that an object's address be determined when the capability is first used.

19.5 Security Attacks

Recent cyber attacks on e-businesses have made the front pages of newspapers worldwide. Denial-of-service attacks (DoS), viruses and worms have cost companies billions of dollars and caused countless hours of frustration. Many of these attacks allow the perpetrator to penetrate a network or system, which can lead to data theft, data corruption and other attacks. In this section we discuss several types of attacks against computer systems. Section 19.6, Attack Prevention and Security Solutions, discusses possible solutions to protect a computer's information and integrity.

19.5.1 Cryptanalysis

A **cryptanalytic attack** attempts to decrypt ciphertext without possessing the decryption key. The most common form of cryptanalytic attacks are those in which the encryption algorithm is analyzed to find relations between bits of the encryption key and bits of the ciphertext. The goal of such an attack is to determine the key from the ciphertext.

Weak statistical trends between ciphertext and keys can be exploited to gain knowledge about the key. Proper key management and key expiration dates can reduce susceptibility to cryptanalytic attacks. The longer an encryption key is used, the more ciphertext an attacker can use to derive the key. If a key is covertly recovered by an attacker, it can be used to decrypt every message that uses the key.

Self Review

1. How do key expiration dates reduce the effectiveness of cryptanalytic attacks?

Ans: 1) The longer an encryption key is used, the more ciphertext an attacker can use to derive the key. Key expiration addresses this concern by limiting how long each key is in use.

19.5.2 Viruses and Worms

A **virus** is executable code—often sent as an attachment to an e-mail message or hidden in files such as audio clips, video clips and games—that attaches to or overwrites other files to replicate itself. Viruses can corrupt files, control applications or even erase a hard drive. Today, viruses can be spread across a network simply by sharing "infected" files embedded in e-mail attachments, documents or programs.

A **worm** is executable code that spreads by infecting files over a network. Worms rarely require any user action to propagate, nor do they need to be attached to another program or file to spread. Once a virus or worm is released, it can spread rapidly, often infecting millions of computers worldwide within minutes or hours.

Viruses can be classified as follows:

1. **boot sector virus**—infects the boot sector of the computer's hard disk, allowing it to load with the operating system and potentially control the system.

2. **transient virus**—attaches itself to a particular computer program. The virus is activated when the program is run and deactivated when the program is terminated.
3. **resident virus**—once loaded into the memory of a computer, it operates until the computer is powered down.
4. **logic bomb**—executes its code, or **payload**, when a given condition is met. An example of a logic bomb is a **time bomb**, which is activated when the clock on the computer matches a certain time or date.

A **Trojan horse** is a malicious program that hides within a trusted program or simulates a legitimate program or feature, while actually causing damage to the computer or network when the program is executed. The name Trojan horse originates from the story of the Trojan War in Greek legend. [Note: In this story, Greek warriors hid inside a wooden horse, which the Trojans took within the walls of the city of Troy. When night fell and the Trojans were asleep, the Greek warriors came out of the horse and opened the gates to the city, letting the Greek army enter and destroy the city of Troy] Trojan horse programs can be particularly difficult to detect, because they appear to be legitimate and operational applications.

Back-door programs are resident viruses that allow the sender complete, undetected access to the victim's computer resources. These types of viruses are especially threatening to the victim, as they can be programmed to log every keystroke (capturing all passwords, credit card numbers, etc.).

Widespread Viruses

Two viruses that attracted significant media attention are Melissa, which struck in March 1999, and the ILOVEYOU virus that hit in May 2000. Each caused billions of dollars of damage. The Melissa virus spread in Microsoft Word documents sent via e-mail. When the document was opened, the virus was triggered, at which point it accessed a user's Microsoft Outlook address book (i.e., a list of e-mail addresses) on that computer and sent the infected Word attachment by e-mail to a maximum of 50 people in the user's address book. Each time another user opened the attachment, the virus would send up to 50 additional messages. Once resident in a system, the virus also infected any files saved using Microsoft Word.

The ILOVEYOU virus was sent as an attachment to an e-mail posing as a love letter. The message in the e-mail said "Kindly check the attached love letter coming from me." Once opened, the virus accessed the Microsoft Outlook address book and sent messages to each address listed, enabling the virus to spread rapidly worldwide. The virus corrupted many types of files, including operating system files. Many networks were disabled for days due to the massive number of e-mails generated.

This virus exposed e-mail security inadequacies, such as a lack of software to scan file attachments for security threats before they are opened. It also taught users to be more aware of suspicious e-mails, even those from someone with whom they are familiar.

Sapphire/Slammer Worm: Analysis and Implications

Worms spread by exploiting weaknesses in communications channels established by software, either by applications or by the operating system. Once a weakness is discovered, a worm can produce network traffic sufficient to disable a single computer or a network of computers. Further, a worm can be designed to execute code on the computer it infects, potentially allowing the worm's creator to obtain or destroy sensitive information.

Worm attacks that have received media attention include Nimda, Code Red and Sapphire, also called Slammer. Slammer, which infected most vulnerable computers within 10 minutes of its release on January 25, 2003, doubled the number of computers it infected every 8.5 seconds. [Note: Vulnerable computers were those running Microsoft SQLServer 2000 to which a security patch, released by Microsoft in July 2002, was not applied.] The infection rate was two orders of magnitude faster than its notable predecessor, the Code Red worm. The latter, a 4KB worm, instantiated multiple threads to create TCP connections to infect new hosts. On the contrary, Slammer operated over UDP and its payload was contained in a single 404-byte UDP packet. The connectionless UDP protocol, coupled with a random-scanning algorithm to generate target IP addresses, made the Slammer worm particularly virulent. A **random-scanning algorithm** uses pseudorandom numbers to generate a broad distribution of IP addresses as targets to infect.

The Slammer worm caused system and network outages due to network saturation by its UDP packets. Interestingly, the worm included no malicious payload and attacked a vulnerability in an application of relatively limited worldwide use. The Slammer worm also contained what appeared to be a logic error in its random-scanning algorithm that significantly limited the number of IP addresses the worm could reach.⁴⁰

Antivirus software can protect against viruses and some worms. Most antivirus software is reactive, meaning that it can attack known viruses, rather than protecting against unknown or future viruses. We discuss antivirus software in Section 19.6.3.

Self Review

1. How do worms differ from other viruses?
2. What weaknesses in computer systems did the Melissa and ILOVEYOU viruses expose?

Ans: 1) Worms typically spread via network connections and do not require user interaction to spread. 2) The viruses exposed insufficient file-scanning antivirus software for e-mail applications and demonstrated that users did not treat suspicious e-mails with appropriate care.

19.5.3 Denial.of.Service (DoS) Attacks

A **denial-of-service (Dos) attack** prevents a system from servicing legitimate requests. In many DoS attacks, unauthorized traffic saturates a network's resources,

restricting access for legitimate users. Typically, the attack is performed by flooding servers with data packets. Denial-of-service attacks usually require a network of computers to work simultaneously, although some skillful attacks can be achieved with a single machine. Denial-of-service attacks can cause networked computers to crash or disconnect, disrupting service on a Web site or even disabling critical systems such as telecommunications or flight-control centers.

Another type of denial-of-service attack targets a network's routing tables. Recall that routing tables provide a view of the network topology and are used by a router to determine where to send data. This type of attack is accomplished by modifying the routing tables, thus maliciously redirecting network activity. For example, the routing tables can be changed to send all incoming data to one address in the network. A similar attack, called a **DNS (domain name system) attack**, can modify the address to which network traffic for a particular Web site is sent. Such attacks can be used to redirect users of a particular Web site to another, potentially malicious, Web site. These attacks are particularly dangerous if the illegitimate Web site poses as the real one, leading users to disclose sensitive information to the attacker.

In a **distributed denial-of-service attack**, packet flooding comes from multiple computers at once. Such attacks are typically initiated by an individual who has infected several computers with a virus to gain unauthorized access to them to carry out the attack. Distributed denial-of-service attacks can be difficult to stop, because it is not easy to determine which requests on a network are from legitimate users and which are part of the attack. It is also particularly difficult to identify the perpetrator of such attacks, because they are not carried out directly from the attacker's computer.

In February 2000, distributed denial-of-service attacks shut down a number of high-traffic Web sites, including Yahoo!, eBay, CNN Interactive and Amazon. In this case, a single user utilized a network of computers to flood the Web sites with traffic that overwhelmed the sites' computers. Although denial-of-service attacks merely shut off access to a Web site and do not affect the victim's data, they can be extremely costly. For example, when eBay's Web site went down for a 24-hour period on August 6, 1999, its stock value declined dramatically.⁴¹

Who is responsible for viruses and denial-of-service attacks? Most often the responsible parties are referred to as **hackers**, but really that is a misnomer. In the computer field, hacker refers to an experienced programmer, often one who programs as much for personal enjoyment as for the functionality of the application. The true term for such people is **cracker**, which is someone who uses a computer maliciously (and often illegally) to break into another system or cause another system to fail.

Self Review

1. How are DNS attacks harmful?
- 2.. Why is it difficult to detect and stop distributed denial-of-service attacks?

Ans: 1) Users unaware of malicious activity could submit sensitive information to a cracker. Legitimate Web sites may lose revenue because users are redirected to alternate sites. 2) It may be difficult to distinguish between legitimate and malicious users.

19.5.4 Software Exploitation

Another problem plaguing e-businesses is software exploitation by crackers. Every program on a networked machine should be checked for vulnerabilities. However, with millions of software products available and vulnerabilities discovered daily, this becomes an overwhelming task. One common vulnerability exploitation method is a **buffer overflow**, in which a program receives input that is larger than its allocated space.

A buffer overflow occurs when an application sends more data to a buffer than it can hold. A buffer overflow attack can push the additional data into adjacent buffers, corrupting or overwriting existing data. A well-designed buffer overflow attack can replace executable code in an application's stack to alter its behavior. Buffer overflow attacks may contain malicious code that will then be able to execute with the same access rights as the application it attacked. Depending on the user and application, the attacker may gain access to the entire system. BugTraq (www.securityfocus.com) was created in 1993 to list vulnerabilities, how to exploit them and how to repair them.

Self Review

1. Why are buffer overflow attacks dangerous?
2. How can buffer overflow attacks be prevented?

Ans: 1) Buffer overflow attacks may modify an application's stack so that an attacker can execute code to gain access to the entire system. 2) An application can rigidly enforce buffer limits when accepting input. Also, the system can implement stack segments that are not executable.

19.5.5 System Penetration

The result of many security attacks is system or network penetration. According to a study by the Computer Security Institute (www.gocsi.com), 40 percent of the respondents reported that an outsider had successfully penetrated their systems.⁴² After an attacker exploits an operating system or the software running on the computer, the system is vulnerable to a number of attacks—ranging from data theft and manipulation to a system crash. A **system penetration** is a successful breach of computer security by an unauthorized external user.⁴³ Every system penetration is potentially dangerous, although a quick response can usually thwart an intruder's attack before any significant damage is done. Many attacks, such as data theft and Web defacing, rely on a successful system penetration as a foundation.

Web defacing is a popular form of attack wherein the crackers illegally obtain access to modify an organization's Web site and change the contents. Web defacing has attracted significant media attention. A notable case occurred in 1996, when Swedish crackers changed the Central Intelligence Agency Web site to read "Central Stupidity Agency." The vandals placed obscenities, political messages, notes to system administrators and links to adult-content sites on the page. Many other popular and large Web sites have been defaced. Defacing Web sites has become over-

whelmingly popular among crackers today, causing archives of attacked sites (with records of more than 15,000 vandalized sites) to close because of the volume of sites that were vandalized daily.⁴⁴

System penetration often occurs as a result of a Trojan horse, back-door program or an exploited bug in software or the operating system. Allowing external users to access applications via the Web provides another channel for an attacker to penetrate a system.⁴⁵ Vulnerabilities in common Web server applications, such as Microsoft Internet Information Services (IIS) and Apache HTTP Server, give attackers a well-known route to penetrating a system if administrators fail to apply necessary patches. System penetration can also occur on personal computers through software connected to the Internet, such as Web browsers.

The CERT®/CC (Computer Emergency Response Team Coordination Center; www.cert.org) at Carnegie Mellon University's Software Engineering Institute is a federally funded research and development center. CERT/CC responds to reports of viruses and denial-of-service attacks and provides information on network security, including how to determine if a system has been compromised. The site provides detailed incident reports of viruses and denial of service attacks, including descriptions of the incidents, their impact and solutions. The site also reports the vulnerabilities in popular operating systems and software packages. The CERT Security Improvement Modules are excellent tutorials on network security. These modules describe the issues and technologies used to solve network security problems.

Self Review

1. List several techniques crackers use to penetrate systems.
2. Compare Web defacing to a DNS attack.

Ans: 1) Crackers can issue a Trojan horse back-door program, place an exploited bug in software or the operating system or access applications via an organization's Web site. 2) Web defacing occurs when crackers illegally obtain access to modify an organization's Web site and change its contents. DNS attacks modify the address to which network traffic for a particular Web site is sent, which can redirect users of a particular Web site to another.

19.6 Attack Prevention and Security Solutions

The previous section detailed several common attacks against computer security. Though the number of threats to computer security may seem overwhelming, in practice, common sense and diligence can prevent a large number of attacks. To further bolster security, additional hardware and software that specializes in thwarting a variety of attacks can be installed on computers and in networks. This section describes several common security solutions.

19.6.1 Firewall

A **firewall** protects a local area network (LAN) from intruders outside the network. A firewall polices inbound and outbound traffic for the LAN. Firewalls can prohibit all data transmission that is not expressly allowed, or allow all data transmission that

is not expressly prohibited. The choice between these two models can be determined by the network security administrator; the former provides a high level of security, but might prevent legitimate data transfer. The latter leaves the system more susceptible to attack, but generally does not restrict legitimate network transfer. Each LAN can be connected to the Internet through a gateway, which typically includes a firewall. For years, the most significant security threats originated from employees inside the firewall. Now that businesses rely heavily on access to the Internet, an increasing number of security threats originate outside the firewall—from the hundreds of millions of people connected to the company network via the Internet.⁴⁶

There are two primary types of firewalls. A **packet-filtering firewall** examines all data sent from outside the LAN and rejects data packets based on predefined rules, such as reject packets that have local network addresses or reject packets from certain addresses or ports. For example, suppose that a hacker from outside the network obtains the address of a computer inside the network and attempts to pass a harmful data packet through the firewall by sending a packet indicating that it was sent from the computer inside the network. In this case, a packet-filtering firewall will reject the data packet because the return address of the inbound packet has clearly been modified. A limitation of packet-filtering firewalls is that they consider only the source of data packets; they do not examine the attached data. As a result, malicious viruses can be installed on an authorized user's computer, allowing an attacker access to the network without that user's knowledge. The goal of an **application-level gateway** is to protect the network against the data contained in packets. If the message contains a virus, the gateway can block it from being sent to the intended receiver.

Installing a firewall is one of the most effective and easiest ways to add security to a small network.⁴⁷ Often, small companies or home users who are connected to the Internet through permanent connections, such as cable modems, do not employ strong security measures. As a result, their computers are prime targets for crackers to exploit for denial-of-service attacks or information theft. However, it is important for all computers connected to the Internet to contain some degree of security for their systems. Many popular network products, such as routers, provide firewall capabilities and certain operating systems, such as Windows XP, provide software firewalls. In fact, the Windows XP Internet Connection Firewall (ICF) is enabled by default. Section 21.13.3, Internet Connection Firewall, discusses how Windows XP implements its firewall. Numerous other firewall software products are available; several are listed in the Web Resources section at the end of the chapter.

Air gap technology is a network security solution that complements the firewall. It secures private data from external traffic accessing the internal network. The air gap separates the internal network from the external network, and the organization decides which information will be made available to external users. Whale Communications created the e-Gap System, which is composed of two computer servers and a memory bank. The memory bank does not run an operating system; therefore hackers cannot take advantage of common operating system weaknesses to access network information.

Air gap technology does not allow outside users to view the network's structure, which prevents hackers from searching the network for weaknesses. The e-Gap Web Shuttle feature allows safe external access by restricting the system's "back office," where an organization's most sensitive information and IT-based business processes are controlled. Users who wish to access a network hide behind the air gap, where the authentication server is located. Authorized users gain access through a single sign-on capability, allowing them to use one login password to access authorized areas of the network.

The e-Gap Secure File Shuttle feature moves files in and out of the network. Each file is inspected behind the air gap. If the file is deemed safe, it is carried by the File Shuttle into the network.⁴⁸

Air gap technology is used by e-commerce organizations to allow their clients and partners to access information with transparent security, thus reducing the cost of inventory management. Military, aerospace and government industries, which store highly sensitive information, employ air gap technology.

Self Review

1. Is a home user more likely to use a firewall to prohibit all data flow not expressly allowed or to allow all data that is not expressly prohibited?
2. Discuss the difference between packet-filtering firewalls and application gateways.

Ans: 1) A home user would be more likely to allow all data that is not expressly prohibited, because the user will most likely want to access a large (and evolving) set of Web sites, located at different IP addresses. 2) A packet-filtering firewall examines all data sent from outside the LAN and rejects data packets based on predefined rules, such as reject packets that have local network addresses or reject packets from certain addresses or ports. An application-level gateway protects the network against the data contained in packets. If the message contains a virus, the gateway can block it from being sent to the intended receiver.

19.6.2 Intrusion Detection Systems (IDSs)

Intrusion detection systems (IDSs) monitor networks and application **log files**, which record information about system behavior, such as the time at which operating system services are requested and the name of the process that requests them. IDSs examine log files to alert system administrators of suspicious application and/or system behavior. If an application exhibits erratic or malicious behavior, an IDS can halt the execution of that process.⁴⁹

Host-based intrusion detection systems monitor system and application log files, which is especially useful for detecting Trojan horses. **Network-based intrusion detection** software monitors traffic on a network for any unusual patterns that might indicate DoS attacks or access to a network by an unauthorized user. System administrators can then check their log files to determine if there was an intrusion and, if so, track the offender. Intrusion detection products are commercially available from companies such as Cisco (www.cisco.com/warp/public/cc/pd/sqsw/sqidsz), Hewlett-Packard (www.hp.com/security/home.html) and Symantec

(www.symantec.com). There is also an open-source network intrusion detection system from Snort (www.snort.org).

Intrusion detection via **static analysis** attempts to detect when applications have been corrupted by a hacker. The static analysis technique assumes that hackers attempt to attack a system using system calls. Under this assumption, the first step in detecting intrusions is to build a model of an application's expected behavior (i.e., a pattern of system calls the application typically generates). The application's pattern of system calls is then monitored as it runs; an attack can be detected if this pattern differs from the static model.⁵⁰

The **OCTAVESM (Operationally Critical Threat, Asset and Vulnerability Evaluation) method**, developed at the Software Engineering Institute at Carnegie Mellon University, evaluates a system's security threats. There are three phases in OCTAVE: building threat profiles, identifying vulnerabilities, and developing security solutions and plans. In the first stage, the organization identifies its important information and assets, then evaluates the levels of security required to protect them. In the second phase, the system is examined for weaknesses that could compromise the valuable data. The third phase is to develop a security strategy as advised by an analysis team of three to five security experts assigned by OCTAVE. This approach is one of the first of its kind, in which the owners of computer systems not only obtain professional analysis, but also participate in prioritizing the protection of crucial information.⁵¹

Self Review

1. Name a major drawback to IDSs.
2. Explain the difference between host-based and network-based IDSs.

Ans: 1) IDSs cannot detect intrusion with perfect accuracy. Thus, an IDS may prevent an authorized user from performing a legitimate operation and may enable an intrusion to go undetected. 2) Host-based IDSs monitor the current system's log files. Network-based IDSs monitor packets that travel over a network.

19.6.3 Antivirus Software

As discussed in Section 19.5.2, viruses and worms have become a menace to businesses and home users alike and have cost businesses billions of dollars.⁵² The number of reported viruses has increased steadily since the mid-1990s.⁵³ In response, antivirus software has been developed and modified to meet the increasing number and variety of virus attacks on computer systems. **Antivirus software** attempts to protect a computer from a virus and/or identify and remove viruses on that computer. There are a variety of techniques antivirus software may use to detect and remove viruses from a system; however, none can offer complete protection.

Signature-scanning virus detection relies on knowledge about the structure of the computer virus's code. For example, many antivirus programs maintain a list of known viruses and their code. All viruses contain a region called the **virus signature** that does not change as the virus spreads. In practice, most known virus lists main-

tain a list of virus signatures. In this case, virus detection software scans the computer, comparing file data to virus code.

A weakness of known virus lists is that they could become prohibitively large as viruses proliferate. The virus list must be updated regularly to successfully identify emerging viruses. Perhaps the most serious weakness of known virus lists is that they can recognize only viruses that have been previously identified by the list provider. Thus, known virus lists generally do not protect against new and unidentified viruses.

A known virus list can be particularly ineffective against variants and polymorphic viruses. A **variant** is a virus whose code has been modified from its original form, yet still retains its malicious payload. A **polymorphic virus** changes its code (e.g., via encryption, substitution, insertion, and the like) as it spreads, to evade known virus lists (Fig. 19.8).

Although virus signatures improve a virus scanner's ability to detect viruses and their variants, they introduce the possibility of false positive and false negative virus detection. False positive virus alerts incorrectly indicate that a virus is resident in a file, whereas false negatives incorrectly determine that an infected file is clean. These incorrect results become more frequent as the size of a virus signature

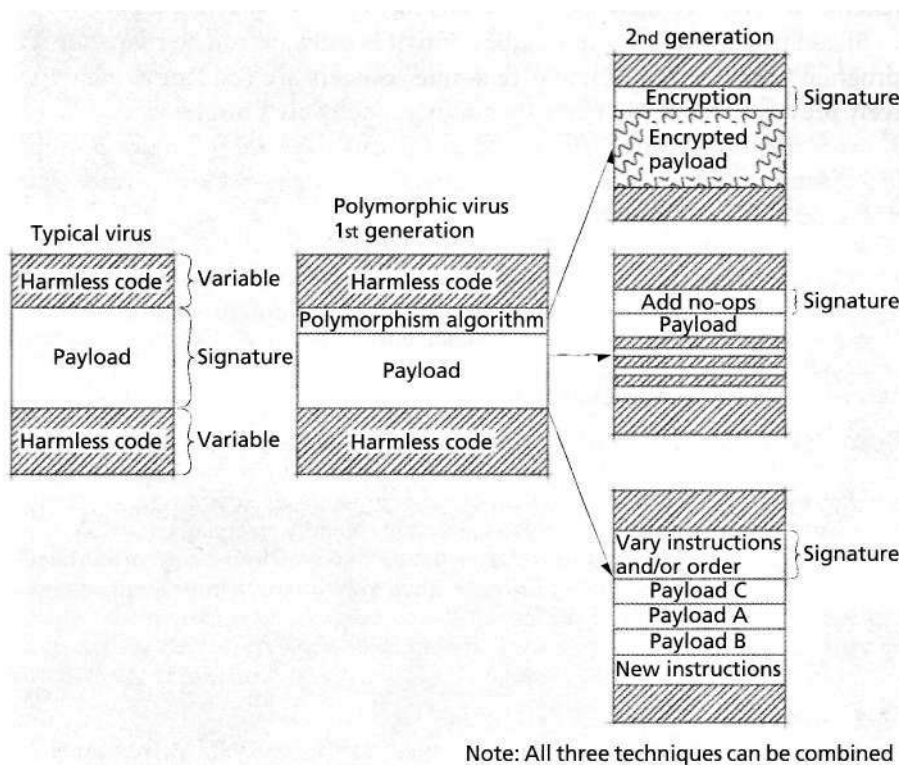


Figure 19.8 | Polymorphic virus.

becomes smaller. The problem of preventing false positive and false negative readings when scanning for viruses is a challenging one, as it has been demonstrated that virus signatures as small as two bytes can be created.^{54, 55}

An alternative to signature scans is **heuristic scanning**. Viruses are characterized by replication, residence in memory and/or destructive code. Heuristic scans can prevent the spread of viruses by detecting and suspending any program exhibiting this behavior. The primary strength of heuristic scanning is that it can detect viruses that have not yet been identified. Similar to signature scanning, however, heuristic scanning is also susceptible to false reporting. Most antivirus software employs a combination of signature and heuristic scanning.

All viruses (with the exception of worms) must modify a file to infect a computer. Consequently, another antivirus technique is to monitor changes to files by checking file consistency. Most consistency checks are implemented as checksums of protected files. In a system that experiences a high volume of file I/O, maintaining a record of file consistency is not a viable option if users expect fast response times. Consequently, many antivirus programs ensure file consistency for a limited set of files (typically the operating system files). However, file consistency scans cannot protect against viruses resident on the computer before antivirus software installation. Some operating systems, such as Windows XP, perform consistency checks on vital system files and replace them if they are altered, to protect system integrity.⁵⁶

In addition to scanning techniques, antivirus software can be characterized by its program behavior. For example, **real-time scanners** are resident in memory and actively prevent viruses, whereas other antivirus software must be loaded manually and serves only to identify viruses. Some antivirus software programs prompt the user for action when a virus is detected, whereas others remove viruses without user interaction.⁵⁷

Self Review

1. Does signature scanning or heuristic scanning provide better protection against new .. unidentified viruses?
2. Describe several weaknesses of known virus lists.

Ans: 1) Heuristic scanning provides better protection against new and unidentified viruses because it can detect new virus behavior even when its signature is not available. **2)** A weakness of known virus lists is that they could become prohibitively large as viruses proliferate. The virus list must be updated regularly to successfully identify emerging viruses. Perhaps the most serious weakness of known virus lists is that they can recognize only viruses that have been previously identified by the list provider. Thus, known virus lists generally do not protect against new and unidentified viruses.

19.6.4 Security Patches

Operating systems and other software often contain security flaws that are not discovered until after many users have installed them. To address security flaws in a timely manner, the software developers must:

- discover previously unknown flaws,
- quickly release patches for the flaws and
- establish strong lines of communication with users.

To reduce the damage caused by security flaws, developers should address security flaws that have been exploited and actively search and fix those that have not. A code release that addresses a security flaw is often called a **security patch**.

Often, simply releasing a patch for a security flaw is insufficient to improve security. For example, the Slammer worm (Section 19.5.2, Viruses and Worms) exploited a security flaw for which a patch had been released six months earlier. Thus software developers should address security flaws by notifying their users quickly and providing software that facilitates the process of applying security patches.

OpenBSD is a BSD UNIX project whose primary goal is to create the most secure (UNIX-compatible) operating system (see the Mini Case Study, OpenBSD). The project includes an audit team that continuously searches for security flaws, a full-disclosure policy that describes each security flaw in a public forum and a team that releases security patches quickly. Users must subscribe to a mailing list or visit a Web page to learn of new security flaws and download patches.⁵⁸ Both the OpenBSD and Apple MacOS X operating systems close all ports and disable all network services by default to improve security (see the Mini Case Study, Macintosh).

Microsoft's Trustworthy Computing Initiative has resulted in a steady stream of security patches, called hotfixes, for the Windows line of operating systems. Microsoft offers free software called Automatic Updates that executes in the background of a Windows system to periodically determine if new hotfixes are available. If the Automatic Updates determines that a "critical update" such as a security patch is available, it will display a dialog to prompt the user to download and install the hotfix.⁵⁹ This software also enables critical updates to be installed without user interaction, which can improve system security.

Self Review

1. Why is it important to establish strong lines of communication between software developers and users when releasing security patches?
2. Why is it insufficient to address security flaws only after they are exploited?

Ans: 1) Software developers should address security flaws by notifying their users quickly and providing software that facilitates the process of applying security patches, so that users can protect themselves against known security flaws. 2) Some exploit actions of security flaws not detected by developers and could compromise system security and go undetected for days, months or even years.

19.6.5 Secure File Systems

Access to system resources, including data stored in a file system, can be regulated by the operating system's access control policy. However, such mechanisms do not nec-

essarily prevent access to data stored on the hard drive when it is accessed by a different operating system. Consequently, several of today's operating systems support secure file systems that protect sensitive data regardless of how the data is accessed. Windows XP employs the New Technology File System (NTFS), which protects files via access control and encryption (see Section 21.8, File Systems Management).

The **Encrypting File System (EFS)** uses cryptography to protect files and folders in an NTFS file system. EFS uses secret-key and public-key encryption to secure files.⁶⁰ Each user is assigned a key pair and certificate that are used to ensure that only the user who encrypted the files can access them. File data is lost if the key is lost. EFS is often implemented in multiuser or mobile systems to ensure that protected files would not be accessible to somebody using a stolen or lost machine.



Mini Case Study

OpenBSD

Theo de Raadt, an original member of the NetBSD project, left NetBSD in 1995 to start the OpenBSD project using the open-source NetBSD source code as a base.^{61, 62}

OpenBSD is considered by many to be the most secure OS available.^{63, 64, 65} It has been more than four years (as of the time this book was published) since OpenBSD has been cracked while running with default settings.^{66, 67}

OpenBSD's extraordinary level of security has been achieved primarily by the core team's code-auditing process. Every source file in the system has been analyzed several times by various developers on the core project team—an ongoing process. These audits check for bugs such as classic security holes like buffer overflow opportunities.

Any found bugs are posted immediately to the OpenBSD project site and are generally fixed in less than a day.^{68, 69, 70, 71}

OpenBSD includes several security features, such as built-in support for OpenSSH and OpenSSL (open-source implementations of the SSH and SSL security protocols).⁷² OpenBSD was also the first operating system to implement IPsec, the IP Security protocol widely used in Virtual Private Networks (VPNs), discussed in Section 19.10.2. It also includes the network authentication protocol Kerberos version five.⁷³

The OpenBSD group (www.openbsd.org) ensures that its operating system is secure upon installation by shipping OpenBSD with only minimal services enabled and all ports closed

(to prevent network attacks).^{74, 75}

It is interesting to note that Windows Server 2003 follows the same philosophy—a complete reversal from Microsoft's policy in Windows 2000—to enhance security in environments managed by inexperienced administrators.⁷⁶

To maintain its secure core, OpenBSD provides a limited set of applications, though the OpenBSD team claims it can run programs written for any of the major UNIX-based systems.⁷⁷ There is no GUI, making this system an unlikely option for most desktop users but suitable for servers, which must be secure and generally have little direct interaction with users.⁷⁸

Encryption can be applied to individual files or entire folders; in the latter case, every file inside the folder is encrypted. Applying encryption at the folder level often achieves a higher level of security by preventing programs from creating temporary files in plaintext. File encryption in Linux is discussed in Section 20.7, File Systems.



Mini Case Study

Machintosh

In the late 1970s, Apple Computer was working on two new computers: the Lisa, a high-end personal computer, and the **Macintosh**, a more affordable personal computer.^{79, 80, 81} In 1979, Apple cofounder Steve Jobs (now CEO of Apple) visited Xerox's Palo Alto Research Center (PARC).⁸² During his visit, Jobs saw a demo of the Alto—a personal computer that used a graphical user interface (GUI) and a computer mouse. The Alto, developed at PARC in the 1970s, was designed for personal use, however it was too large and expensive to be practical, therefore it was never sold.

Inspired by the Alto, Jobs incorporated GUIs and the computer mouse into both the Apple Lisa and Macintosh operating systems. The Apple GUIs included some features similar to those of the Alto, plus many original features including drag-and-drop file-icon movement, pull-down menus and windows that redraw automatically when uncovered—

all of which worked using a mouse. The Lisa and Macintosh systems were the first personal computers on the market to include a mouse.⁸³ The Lisa debuted in 1983, but it cost ten thousand dollars and was not successful; Apple discontinued the model two years later.⁸⁴ The Macintosh was announced in 1984 with a famous commercial based on George Orwell's "1984,"⁸⁵ which debuted during the Super Bowl. The Macintosh, which was more space efficient and affordable than the Lisa (it sold for \$2,495), became extraordinarily successful.^{86, 87}

The first Macintosh operating system was System 1. Although it had GUI capabilities, it was a fairly rudimentary operating system—it had no virtual memory, did not support multiprogramming and had a single-level file system.⁸⁸ These features were updated in later versions; a hierarchical file structure was implemented in System 3, basic

multiprogramming was added in System 4 and virtual memory was introduced in System 7 which was released in 1990.⁸⁹ In 1997, after numerous bug fixes and application upgrades, version 7.6 was released and the operating system was renamed **Mac OS**.^{90, 91}

Mac OS was completely rebuilt in 2001 for Mac OS X, with the new Darwin kernel incorporating the Mach 3 microkernel and BSD UNIX.⁹² Darwin is a complete UNIX environment including the X11 windows system as an option.⁹³ Apple also made Darwin an open-source project to make it more robust and stable.⁹⁴ Mac OS X has followed OpenBSD's (discussed in the OpenBSD mini case study) lead, improving security by shipping with all ports closed and all network services disabled.⁹⁵ Apple also made Mac OS X compatible with the Windows file formats and Windows Exchange mail servers, making it easy for Mac OS X users to integrate into a business environment.⁹⁶

SelfReview

1. When are access control mechanisms insufficient to protect file data?
2. What is the primary risk of implementing an encrypted file system?

Ans: 1) Access control mechanisms are insufficient to protect file data when the storage device is removed from the computer, because it can be accessed by a system that bypasses the existing access control mechanisms. 2) File data is unrecoverable if the encryption key is lost.

19.6.6 Orange Book Security

To evaluate the security features of operating systems, the U.S. Department of Defense (DoD) published a document entitled "Department of Defense Trusted Computer System Evaluation Criteria," also called the **Orange Book**, in December 1985.^{97, 98} This document is still used to define levies of security in operating systems. The Orange Book, originally designed for evaluating military systems, classifies systems into four levels of security protection—A, B, C and D. The lowest level of security is D and the highest is A. The requirements for each level follow.^{99, 100}

- Level D: Any system that does not meet all the requirements in any other levels. Systems categorized as level D are generally insecure.
- Level C: This level contains two sublevels. Level C1 requires the operating system to separate users and data, which means individuals or groups must log in a username or group name and password to use the system. Private information belonging to an individual or a group is secured to prevent other individuals and groups from reading or modifying it. Early versions of UNIX belong to this level. Level C2 supports only individual login with password, which means users cannot log in with a group name. Authorized individuals can access only certain files and programs. Both C1 and C2 allow individuals to control the access to their files and private information, which means they require only discretionary access control. Most operating systems, such as Windows NT, modern UNIX systems and IBM OS/400¹⁰¹ fall into this category.
- Level B: In this level, mandatory access control is required, which means the operating system requires a predefined central permission scheme to determine the permissions assigned to subjects. The creator of an object does not control the permissions for that object. This level contains three sublevels. In addition to the requirements of C2, B1 requires the operating system to contain a predefined central permission scheme and apply sensitivity labels (e.g., "Confidential") on subjects and objects. The access control mechanism must use these sensitivity labels to determine permissions. Operating systems that satisfy B1 requirements include HP-UX BLS¹⁰², SEVMS¹⁰³ and CS/SX¹⁰⁴. B2 requires the communication line between the user and the operating system for authentication to be secure, in addition to the requirements from B1. Operating systems that satisfy B2 requirements include Multics¹⁰⁵ and VSLAN¹⁰⁶. B3 requires all the features present in B2, in addition to implementing protection domains, providing

secure recovery mechanisms (recovery without compromising protection after a system failure) and monitoring all access to subjects and objects for analysis. The XTS-300 operating system¹⁰⁷ satisfies B3 requirements.

- Level A: This level contains two sublevels. A1 requires all the features provided by B3, and it requires that the operating system's security is formally verified. An example of an operating system that satisfies the A1 requirements is the Boeing MLS LAN¹⁰⁸. The requirements for A2 are reserved for future use.

Self Review

1. Does a system that belongs to category C2 need to satisfy the requirements for level B?
2. How does C2 differ from C1?

Ans: 1) No. Level B is higher than level C2, hence level B has more features and requirements. 2) C1 supports both individual and group logins while C2 supports only individual login.

19.7 Secure Communication

E-business has profited from the rapidly rising number of consumers that own computers with Internet connections. However, as the number of online transactions increases, so does the volume of sensitive data transmitted over the Internet. Applications that process transactions require secure connections through which sensitive data can be transmitted. Several methods for providing secure transactions have been developed in recent years. In the sections that follow, we describe some of these techniques.

There are five fundamental requirements for a successful, secure transaction:

- privacy
- integrity
- authentication
- authorization
- nonrepudiation

The **privacy** issue is: How do you ensure that the information you transmit over the Internet has not been captured or passed to a third party without your knowledge?

The **integrity** issue is: How do you ensure that the information you send or receive has not been compromised or altered?

The **authentication** issue is: How do the sender and receiver of a message verify their identities to each other?

The **authorization** issue is: How do you manage access to protected resources on the basis of user **credentials**, which consists of user's identity (e.g., username) and proof of identity (e.g., password)?

The **nonrepudiation** issue is: How do you legally prove that a message was sent or received? Network security must also address the issue of availability: How do we ensure that the network, and the computer systems to which it connects, will operate continuously? In the sections that follow, we discuss

several implementations of secure communication that derive from encryption techniques discussed in Section 19.2.

Self Review

1. For which of the five fundamental requirements for a successful, secure transaction would encryption be useful?
2. Which of the five fundamental requirements deal with user login?

Ans: 1) Encryption is useful for all five of the fundamental requirements for a successful, secure transaction. 2) Authorization and authentication.

19.8 Key Agreement Protocols

Although public-key algorithms offer flexible and reliable security, they are not efficient for sending large amounts of data. Therefore, public-key algorithms should not be considered replacements for secret-key algorithms. Instead, public-key algorithms are most often employed to exchange secret keys securely. The process by which two parties can exchange keys over an unsecure medium is called a **key agreement protocol**.

The most common key agreement protocol is a **digital envelope** (Fig. 19.9). The message is encrypted using a secret key (Step 1), and the secret key is then encrypted using public-key encryption (Step 2). The sender attaches the encrypted secret key to the encrypted message and sends the receiver the entire package, or envelope. The sender could also digitally sign the envelope before sending it to prove the sender's identity to the receiver (covered in Section 19.8.2, Digital Signatures). To decrypt the envelope, the receiver first decrypts the secret key using the receiver's private key. Then, the receiver uses the secret key to decrypt the actual message. Because only the receiver can decrypt the encrypted secret key, the sender can be sure that only the intended receiver can read the message.

Self Review

1. Why are public-key algorithms employed to exchange secret keys?
2. In the digital envelope protocol, why is it inappropriate to encrypt the entire message using public-key encryption?

Ans: 1) Public-key algorithms are efficient for sending small amounts of data, and they prevent attackers from obtaining the secret key while it is transmitted. 2) Public-key encryption is not efficient for sending large amounts of data because it requires significant processing power.

19.8.1 Key Management

Maintaining the secrecy of private keys is essential to the maintenance of cryptographic system security. Most security breaches result from poor key management (e.g., the mishandling of private keys, resulting in key theft) rather than cryptanalytic attacks.¹⁰⁹